

MI PCIE API

Version 3.0

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. MI PCIE 概述	1
1.1. 模块说明	1
1.2. 流程框图	1
1.3. 关键字说明	2
2. PCIe 总线级联	3
2.1. 概述	3
2.2. 数据传输方式	3
2.3. PCIe 总线接口	3
2.4. 级联方式	3
2.5. PCIe 总线 DMA	6
3. RPMSG 说明	7
3.1. 概述	7
3.2. 流程框图	7
3.2.1 Master(RC 端)的流程框图	8
3.2.2 Slave(EP 端)的流程框图	9
3.3. Target_ID 和 PortID	9
4. MI API 参考	10
4.1. MI_PCIE_CreateDevice	10
4.2. MI_PCIE_DestroyDevice	12
4.3. MI_PCIE_CreateChannel	12
4.4. MI_PCIE_DestroyChannel	14
4.5. MI_PCIE_StartChannel	15
4.6. MI_PCIE_StopChannel	16
4.7. MI_PCIE_DmaTask	17
5. MI PCIE 数据类型	19
5.1. MI_PCIE_DMA_BLOCK_MAX	20
5.2. MI_PCIE_DEV	20
5.3. MI_PCIE_CHN	20
5.4. MI_PCIE_WorkMode_e	21
5.5. MI_PCIE_DevAttr_t	21
5.6. MI_PCIE_DmaBlocks_t	22
5.7. MI_PCIE_DmaTask_t	22
5.8. MI_PCIE_ChnAttr_t	23
5.8.1 Target Id 定义	23
6. MI PCIE 返回值	26
7. MI_PCIE 调试信息	27
7.1. cat	27
7.2. echo	29
8. RPMSG IOCTL 参考	31
8.1. SS_RPMSG_CREATE_EPT_IOCTL	31
8.2. SS_RPMSG_DESTROY_EPT_IOCTL	33

8.3.	SS_RPMSG_DEVICES_INFO_IOCTL.....	34
8.4.	SS_RPMSG_QUERY_IOCTL.....	35
8.5.	SS_RPMSG_LOAD_IMAGE_IOCTL.....	37
8.6.	SS_RPMSG_BOOTUP_IOCTL.....	38
8.7.	SS_RPMSG_BOOTUP_DONE_IOCTL.....	40
9.	RPMSG API 参考.....	42
9.1.	read.....	42
9.2.	write.....	43
10.	RPMSG 接口数据类型.....	44
10.1.	RPMSG_MODE_RC_PORT.....	44
10.2.	RPMSG_MODE_EP_PORT.....	45
10.3.	RPMSG_MODE_UNKNOWN.....	45
10.4.	EPT_TYPE_CUSTOMER.....	45
10.5.	EPT_TYPE_SIGMASTAR.....	46
10.6.	EPT_TYPE.....	46
10.7.	EPT_ADDR_MACRO.....	47
10.8.	RPMSG_QUERY_STAGE.....	47
10.9.	RPMSG_STAGE_RTOS.....	47
10.10.	RPMSG_STAGE_LINUX.....	48
10.11.	RPMSG_STAGE_APP.....	48
10.12.	RPMSG_BOOTUP_FLAG_WAIT_APP.....	49
10.13.	struct ss_rpmsg_endpoint_info.....	49
10.14.	struct ss_rpmsg_devices_info.....	50
10.15.	struct ss_rpmsg_query_info.....	51
10.16.	struct ss_rpmsg_image_info.....	52
10.17.	struct ss_rpmsg_bootup_info.....	52
10.18.	struct ss_rpmsg_bootup_done.....	53

1. MI PCIE 概述

1.1. 模块说明

PCIe（Peripheral component interconnect express）是一种通用的高速串行总线接口标准，广泛用于 PC、手持设备、工业等各个领域。

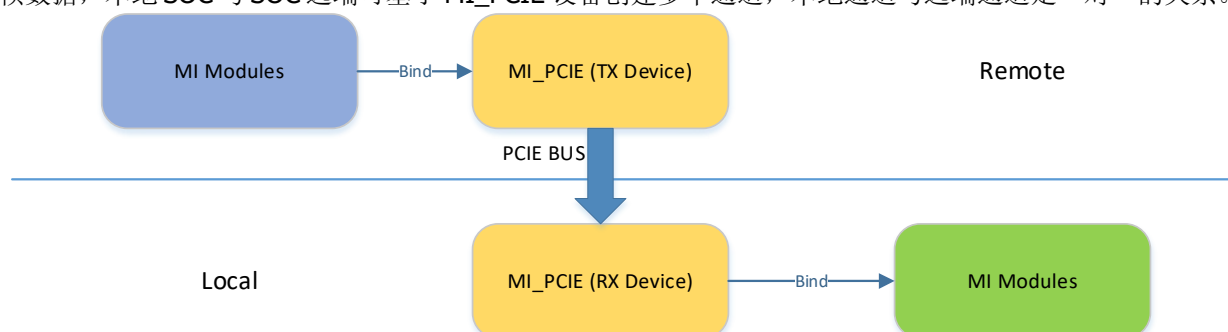
MI_PCIE 具有 MI 通道（channel）和设备（device）的特性，通过其他 MI 模块与 MI_PCIE 模块的通道绑定的方式，可使 MI 内部的数据流经过 PCIe 的总线通讯传输到另外的 SOC 上。

MI_PCIE 另外提供用户层直接操作 PCIe 总线 DMA 的接口用于用户自定义数据类型的传输，并且提供一套基于 PCIe 总线的消息同步机制---RPMSG，消息同步结合 PCIe 总线 DMA 可以使用户扩展基于 PCIe 的传输协议。

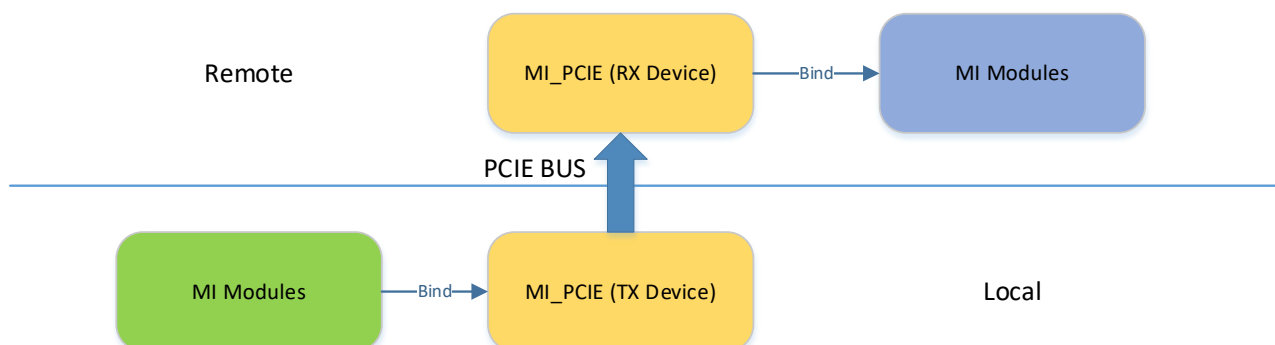
MI_PCIE 的设备数取决于 SOC 上 PCIe 的接口个数，每个接口分配两个设备，分别代表 RX 和 TX 两种模式。RX 设备表示接收 PCIe 总线传来的数据的设备，TX 设备表示发送数据到 PCIe 总线上另外与之级联的 SOC 上，每个设备上可以创建多个通道（channel），通道分时复用 PCIe 总线 DMA。通过 MI_PCIE 提供的接口可以在多个 PCIe 级联的 SOC 间进行数据传输。

1.2. 流程框图

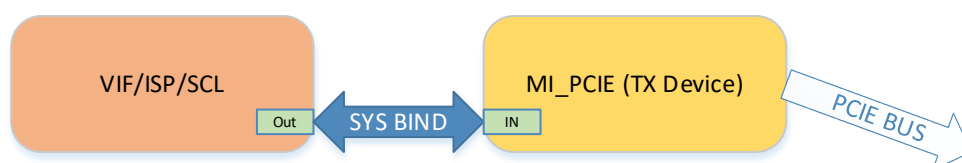
如下图所示，在本地（Local）SOC 端 MI_PCIE 若是 RX 设备，则远端为 TX 设备，本地 RX 设备接收远端 TX 设备传过来的帧数据，本地 SOC 与 SOC 远端可基于 MI_PCIE 设备创建多个通道，本地通道与远端通道是一一对应的关系。



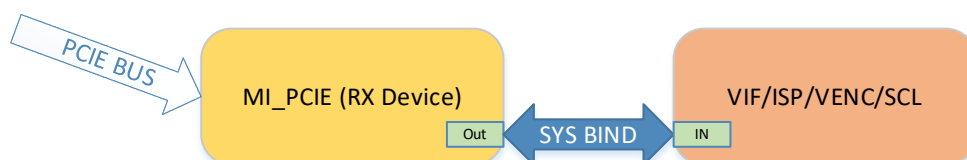
以上情况数据传输方向反过来，本地为 TX 设备，远端为 RX 设备，如下图所示：



当MI_PCIE 设备为TX 模式时，只有一个输入端口（Input Port），类似于MI_DISP 模块，通过MI_SYS 的接口，MI_PCIE TX 设备可以与前一级MI 模块进行帧同步模式（Frame Mode）的绑定，MI_PCIE TX 设备内部通过PCie 总线把帧数据（Frame Data）发送到远端的MI_PCIE RX 设备上，大致流程如下图所示。



远端MI_PCIE 设备为RX 模式，只有一个输出端口（Output port），类似于MI_VIF 模块，通过MI_SYS 的接口，MI_PCIE RX 设备可以与后一级MI 模块进行帧同步模式（Frame Mode）的绑定，把接收到的帧数据（Frame Data）发送给其他MI 模块，流程如下图所示。



MI_PCIE 模块输入和输出端口通过MI_SYS 模块提供的接口与其他模块之间进行绑定，从而解决了帧数据的来源和去向的问题。

1.3. 关键字说明

EP:

PCie 从机工作模式，英文全称 End point

RC:

PCie 主机工作模式，英文全称 Root complex

PCI:

Peripheral component interconnect

PCie:

PCI Express

2. PCIE 总线级联

2.1. 概述

在嵌入式的 SOC 中，除了 CPU 之外，还集成了各种 IP 应用于各种使用场景，在不断提高的需求中，性能会达到瓶颈，例如编解码模块，最大的能力只能到 32 路，若需扩展到 64 路甚至是 128 路，一颗 SOC 实现的设计复杂程度和工艺精度会大大增加，因此从时间和空间的成本上考虑，在短时间内很难研发出具有如此高性能和高集成的 SOC 产品。

在 SOC 上集成 PCIe 高速传输接口是一种很好的解决方案来应对不断增长的性能需求。例如把两颗 SOC 通过 PCIe 总线级联起来，从而能达到性能 1+1 接近于 2 的效果。

PCIe 总线级联并不是一个新概念，AMD 显卡技术曾经使用过 CrossFire 技术通过 PCIe 总线把两个显卡在一个主板上级联起来，实现了 GPU 性能的提升。那么在具有编解码能力和图形图像处理的 SOC 中，基于 PCIe 把两颗 SOC 级联起来，理论上就具有双倍的性能，这取决于 PCIe 的最大传输能力。

2.2. 数据传输方式

MI PCIE 在总线级联的场景下会有两种数据交换的方式，一种是大数据拷贝，需要用到 PCIe 总线上的 DMA，根据目前设计的考虑，仅支持在 EP 端发起 DMA 数据拷贝。

另外一种数据交换是基于 PCIe 之上使用了一套比较通用的 IPC 通信框架：RP Message，主要是用于收发控制信息，基于此 MI_PCIE 在内核态实现了一套 DMA 数据收发的协议用于 MI 绑定数据的传输。RP Message 同时也会有用户层接口，用于扩展给用户实现用户层级的 PCIe 的消息同步的功能，结合 DMA 的用户层接口可以实现某种基于 PCIe 的数据交互协议。

2.3. PCIe 总线接口

主芯片根据规格的不同，有可能会提供两个总线接口，总线接口也称之为 PCIe 端口，每个端口有各自固定的 Port Id，取决于 DTS 中对 PCIe Port 的设定，每个 PCIe 端口都可以独自工作在 RC 或者 EP 模式下，并且有独立的硬件 DMA，一般地，若不是二级级联的场景，主芯片的两个 PCIe 端口都可以设定成 RC 模式，其中一个端口可以用来级联从芯片，另外的端口可以接入一个 PCIe 转 SATA 的转换芯片，用于接入 SATA 设备，MI_PCIE 提供的接口仅用在 PCIe 端口处于芯片级联的模式下 DMA 传输和消息收发，若 PCIe 端口接入其他设备，需要使用标准的 Linux PCIe RC/EP 设备驱动，这部分不在本文档介绍的范围内，请参考 Linux PCIe 驱动的官方文档或者 BSP 相关手册。

2.4. 级联方式

PCIe 级联需要两块或者两块以上的 SOC，在 PCIe 级联的系统中，有一颗 SOC 是工作在主机模式下（RC 模式），而与它连接的 SOC，不管是直接连接还是通过 SWITCH 连接，都属于一级级联的情况，且他们都工作在从机模式（EP 模式）下。

多个 SOC 通过 PCIe 总线连接在一起，连接方式有不同，以下例举几种连接方式：

一个 **RC-TX** 连接一个 **EP-RX**：

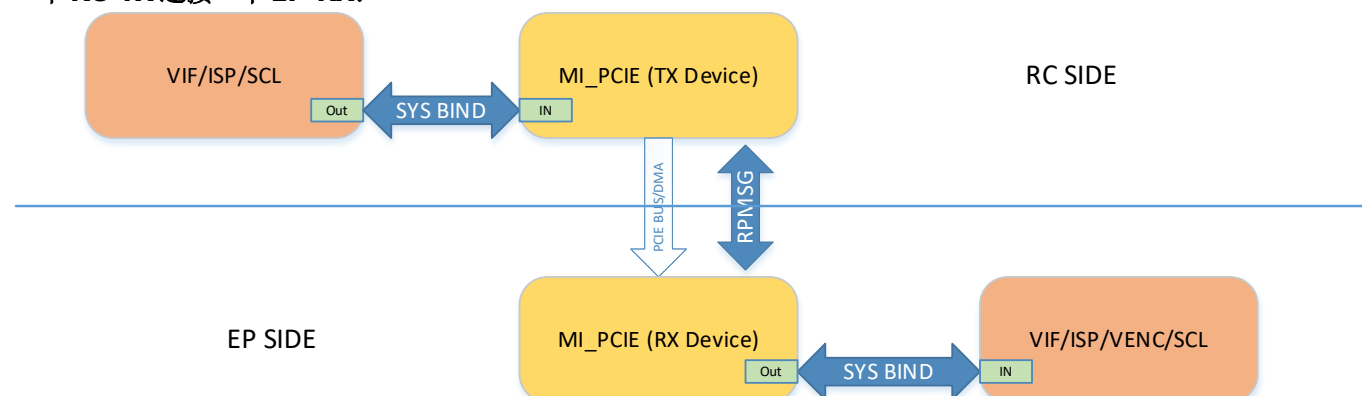


图 2-1 RC 处于 TX 模式

一个 **EP-TX** 连接一个 **RC-RX**：

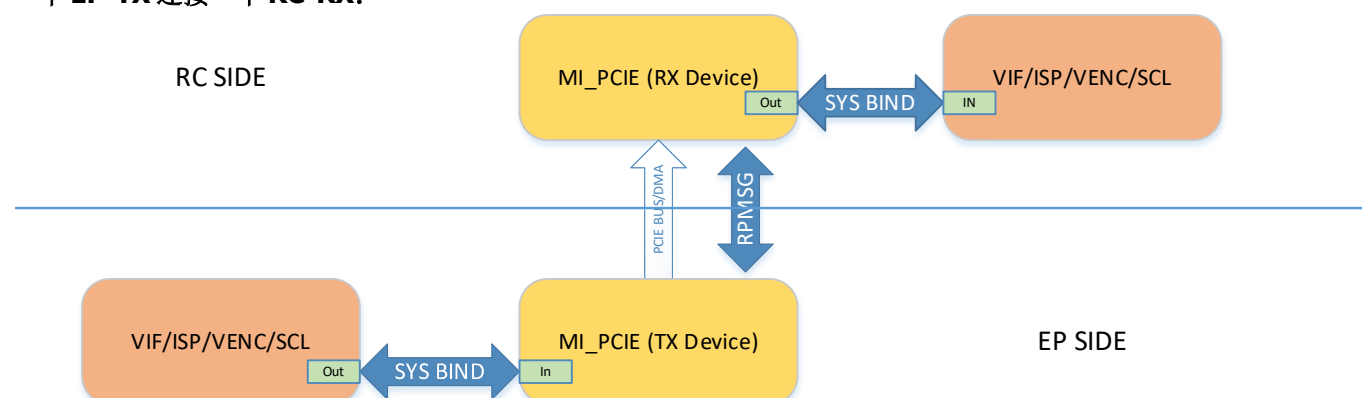
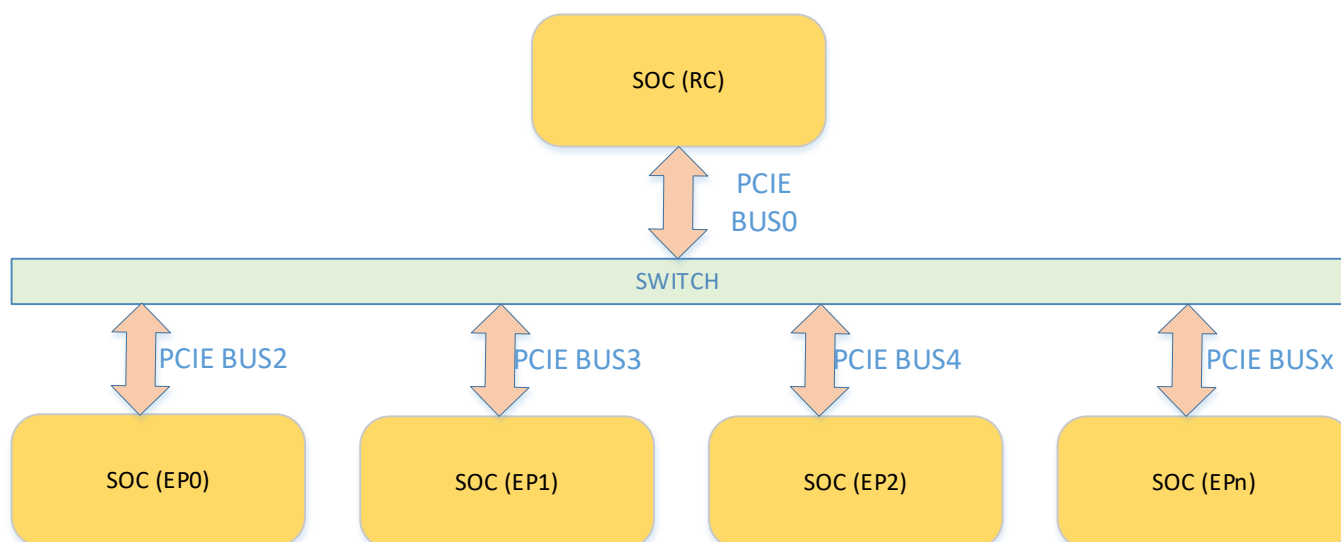


图 2-2 RC 处于 RX 模式

这是最简单的一对一模型，数据传输方向由 TX 往 RX 传输。MI PCIE 模块对设备处于 RX/TX 模式抽象出了两种不同的设备，具体请参考 API，[MI_PCIE_CreateDevice](#)。

一个 **RX** 通过 **SWITCH** 连接多个 **EP**：

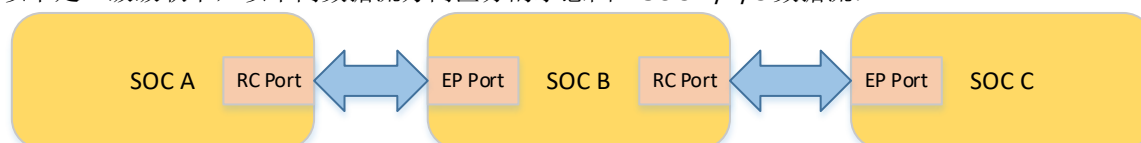
一个 RC 可以通过 SWITCH 连接多个 EP，RC 与 EP 之间可以双向通讯，但是 EP 与 EP 之间不可以直接通讯。连接图如下所示：



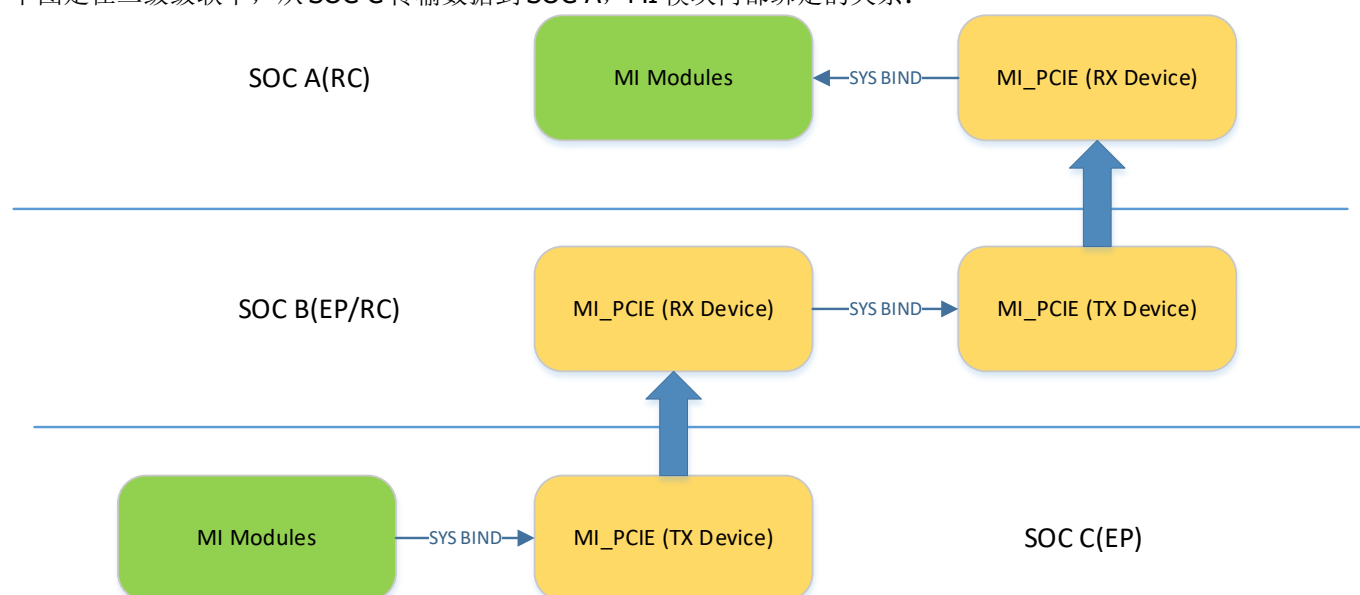
二级级联:

二级级联中 SOC B 会用到两个 PCIe 总线接口，如下图所示 SOC A 访问 SOC C 要通过 SOC B 为 bridge 才能访问。在 SOC B 中与 SOC C 连接的 PCIe 端口会用作 RC 模式。在 SOC B 内部中会通过 MI SYS 的 bind 接口使 PCIe 的两个设备进行串接，从而达到转发的作用。

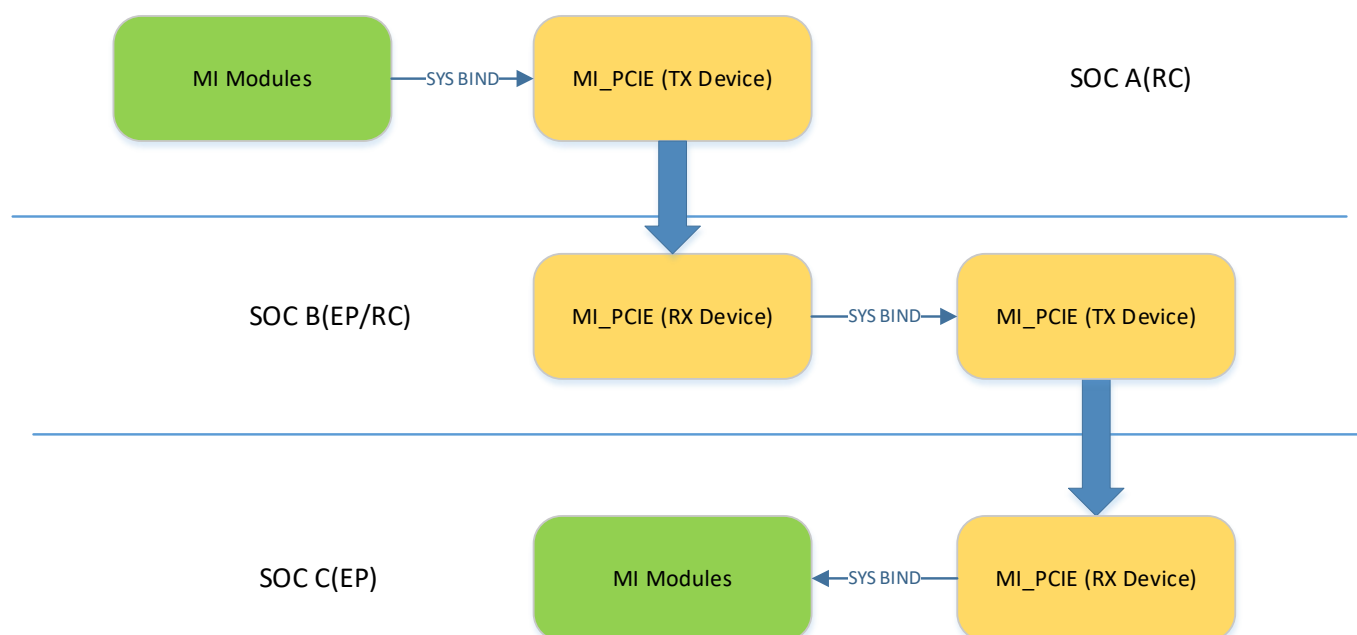
以下是二级级联中，以不同数据流方向区分的示意图。SOC A/B/C 数据流:



下图是在二级级联中，从 SOC C 传输数据到 SOC A，MI 模块内部绑定的关系:



从 SOC A 传输数据到 SOC C，MI 模块内部绑定的关系:



2.5. PCIe 总线 DMA

DMA 是在 PCIe 总线上能够访问本地和远程内存数据的硬件模块。在一级级联的系统中，有且只有一个 PCIe 处于 RC 模式（主机模式），在 RC 模式下会连接多个 EP 端，由于 RC 端访问 EP 端地址空间受到 bar window 的限制，仅使用 bar window 去处理 RC 与 EP 间的 Share memory，这块内存用于 RPMsg 通讯。

在 EP 端 DMA 访问 RC 的地址空间无需做地址空间转换，并且支持双向通讯，因此会限制 MI_PCIE 的用户层 API [MI_PCIE_DmaTask](#) 仅在 EP 端有效，但 RC 和 EP 端都可以向对方发起 RP Message 通讯，因此如果利用此接口在 EP 发起一个 DMA 数据传输任务之前，需要通过 RP Message 获取 RC 端的 MMA 物理地址。

3. RPMSG 说明

3.1. 概述

RPMSG 英文全称是(Remote processor Messaging)，是一种开放式的 processors 间，SoCs 间的通讯协议。Linux Kernel 内置 RPMSG 协议的实现。

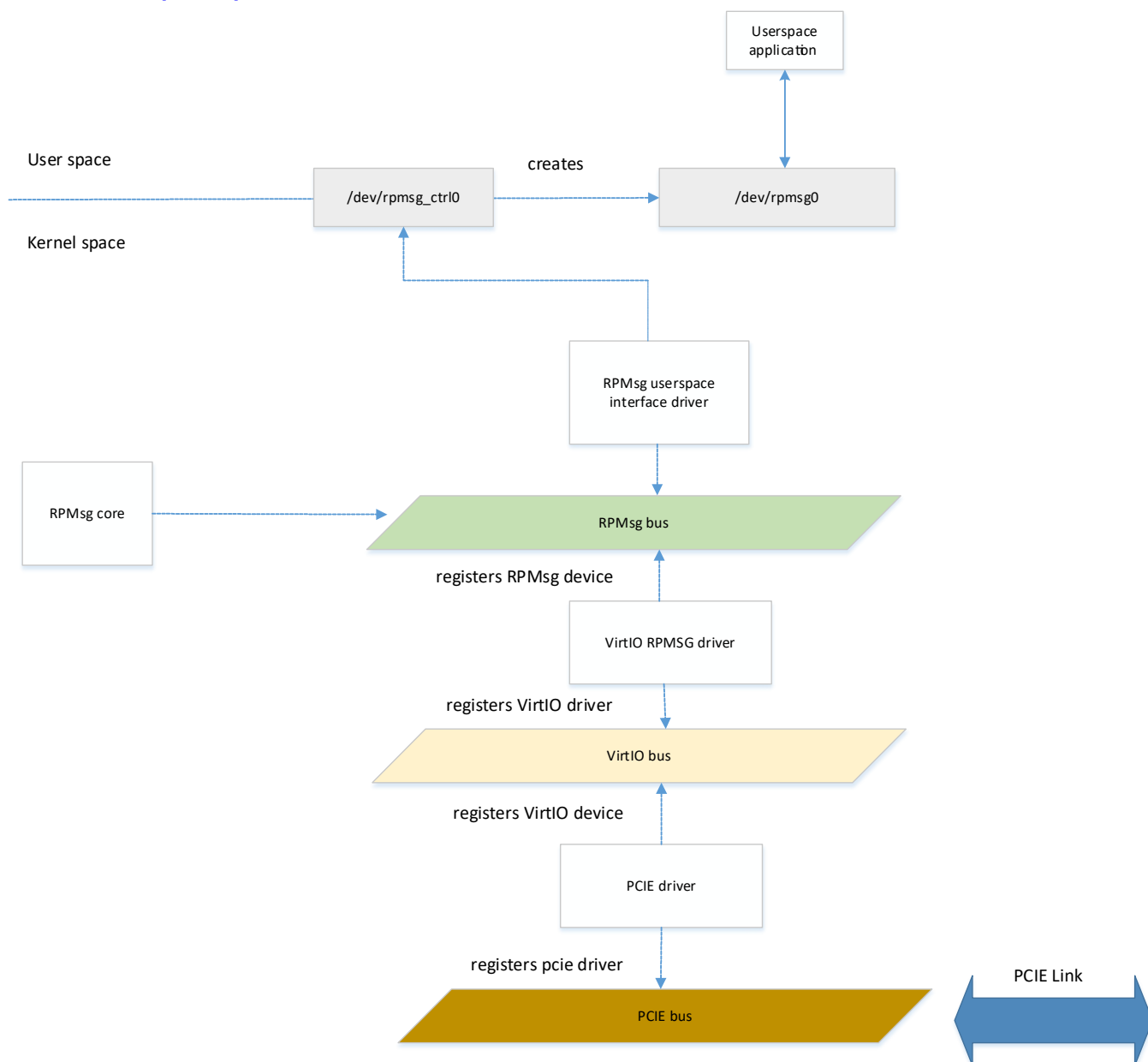
3.2. 流程框图

Sigmastar RPMsg userspace driver 对接 kernel 内置 RPMsg 协议实现，并提供了创建 RPMsg endpoint 的接口，多 SOC 间 userspace app 可以使用这些 endpoint 进行 IPC 通信，同时也提供了引导 PCIe 级联时从片启动的 ioctl 接口给 userspace。

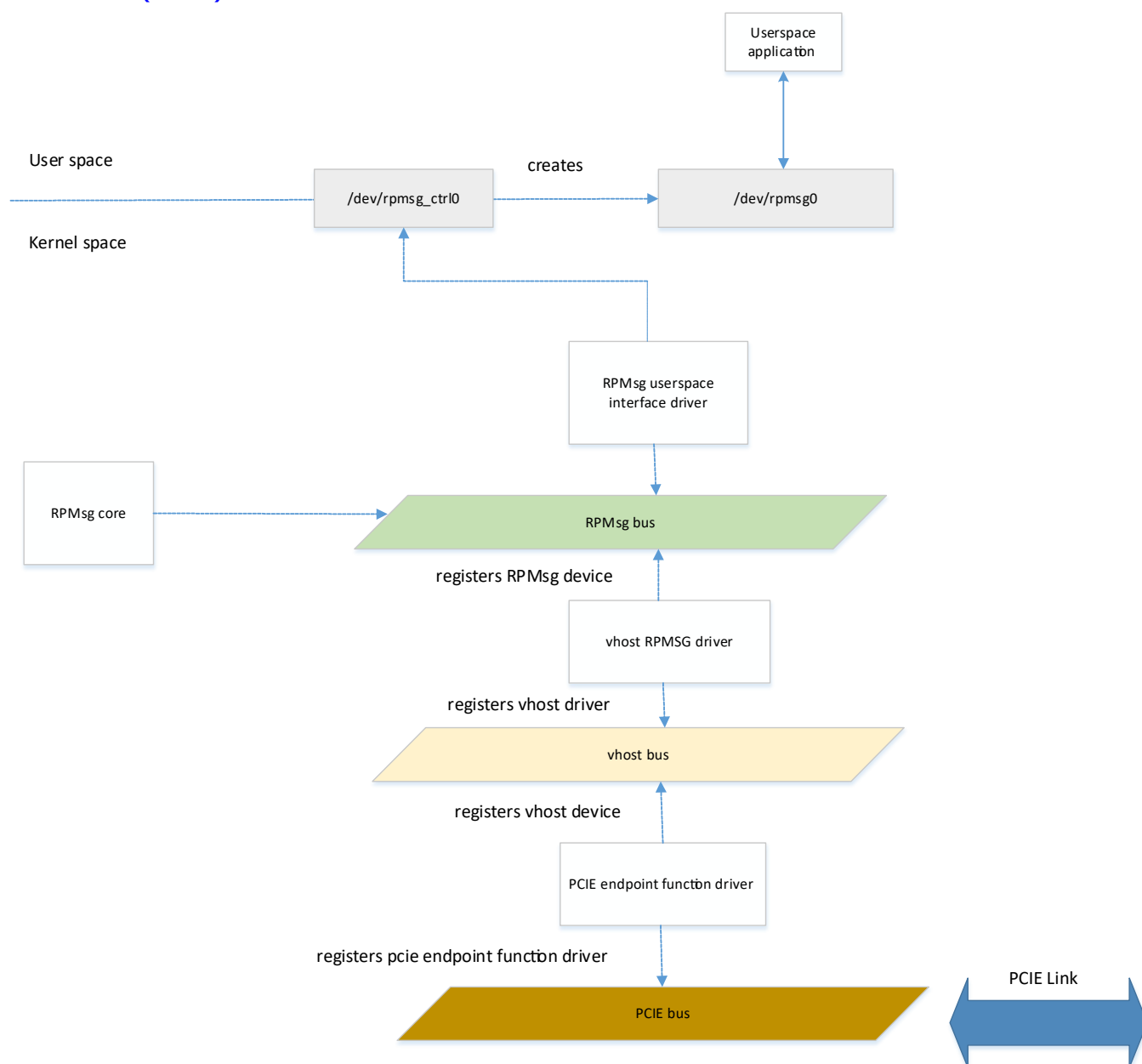
Rpmsg userspace driver 会生成设备节点/dev/rpmsg_ctrl0，userspace app 通过其 ioctl 接口，可以创建用于通信的 Endpoint 设备节点(如:/dev/rpmsg0)，创建节点时，需要指定本地地址和远端的通信地址，如在 SoC A 上指定 src address Src1 和 dst address Dst1，在 SoC B 上指定 src address Dst1 和 dst address Src1，则这样创建的两个 Endpoint 间可以进行通信了。

设备节点/dev/rpmsg_ctrl0 也提供了 ioctl 接口，支持在 RC 端查询连接的从片列表，支持在 EP 端查询当前有连接上主片的 PortID 列表，支持下载 image 到从片 DDR 并启动从片(可等待从片启动完成才返回)等功能。

3.2.1 Master(RC 端)的流程框图



3.2.2 Slave(EP 端)的流程框图



3.3. Target_ID 和 PortID

主片 RC 端要建立和指定从片(EP)通信的 RPMsg Endpoint 时，需要知道对应从片的 **target_id**。我们使用从片所在 PCIe bus number 作为从片的 **target_id**，整个 PCIe 级联方案拓扑结构固定后，从片所在的 PCIe bus number 也是固定的。

从片端要建立和主片通信的 RPMsg Endpoint 时，需要知道从片的哪个 PCIe port 工作在 EP 模式并和要通信的主片 RC 口相连，Muffin SoC 的两个 PCIe port 有固定的 PortID(0 和 1)。

4. MI API 参考

API 名	功能
MI_PCIE_CreateDevice	初始化设备
MI_PCIE_DestroyDevice	去初始化设备
MI_PCIE_CreateChannel	初始化设备通道
MI_PCIE_DestroyChannel	去初始化设备通道
MI_PCIE_StartChannel	启动设备通道传输
MI_PCIE_StopChannel	停止设备通道传输
MI_PCIE_DmaTask	驱动设备传输任务

4.1. MI_PCIE_CreateDevice

➤ 功能

初始化 PCIe 设备，根据设备 id 可以配置 PCIe 设备传输属性，根据设备参数可以配置其模式。

➤ 语法

```
MI_S32 MI_PCIE_CreateDevice(MI\_PCIE\_DEV devId, MI\_PCIE\_DevAttr\_t *pstPcieDevAttr);
```

➤ 形参

参数名称	参数含义	输入/输出
devId	PCIe 设备的 device id。	输入
pstPcieDevAttr	PCIe 的设备属性。	输入

➤ 返回值

返回值 {

- [MI_SUCCESS](#) 成功。
- [MI_ERR_PCIE_INVALID_DEVID](#) 非法的设备 id。
- [MI_ERR_PCIE_BUSY](#) 设备已经创建，或者注册 MI 设备失败
- [MI_ERR_PCIE_ILLEGAL_PARAM](#) 非法的输入参数

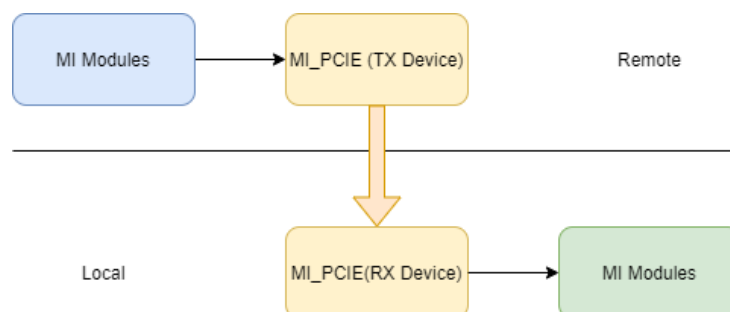
➤ 依赖

- 头文件：mi_sys.h、mi_pcie.h。
- 库文件：无

※ 注意

- 在每个 PCIe 的接口上按照数据传输的方向可使用两个设备，分别表示成 RX/TX 两种属性。设备 ID 为偶数的是 RX 设备，奇数则是 TX 设备。例如在两个 PCIe 接口的 SOC 上，接口 0 上的 PCIe RX 和 TX 对应的设备号为 0、1，在接口 1 上的对应的设备号为 2、3。

- RX、TX 的设备有着不同的特性，如下图所示，若为 RX 设备，只有一个输出 port，若为 TX 设备，只有一个输入 port。MI_PCIE 做为具有 MI 的 Pipeline 属性的一个模块，它可以使用 MI_SYS 提供的接口来参与各个模块间数据的传输，例如使用 MI_SYS 的 get/put/inject/bind 等接口来配置 Frame 数据的通讯。



- MI_PCIE 模块通过 PCIe dma 通讯的方式使本地的 MI Pipeline 与远端的 MI Pipeline 建立连接，请注意其他模块与 MI_PCIE 只支持 E_MI_SYS_BIND_TYPE_FRAME_BASE 方式绑定，且仅支持帧数据（E_MI_SYS_BUFDATA_FRAME）。
- PCIe 的设备的工作模式分为 EP、RC 两种，在同一个 PCIe Port 上的模式必须一致，否则返回 [MI_ERR_PCIE_ILLEGAL_PARAM](#)。

➤ 举例

```

typedef enum
{
    E_MI_PCIE_DEVO_RX,
    E_MI_PCIE_DEVO_TX,
    E_MI_PCIE_DEV1_RX,
    E_MI_PCIE_DEV1_TX,
    E_MI_PCIE_DEV_MAX
}MI_PCIE_DevType_e;
void CreateDestroyPcie(void)
{
    MI_PCIE_DevAttr_t stDevAttr;
    MI_PCIE_DEV devId = (MI_PCIE_DEV)E_MI_PCIE_DEVO_RX;
    MI_S32 s32Ret = 0;

    memset(&stDevAttr, 0, sizeof(MI_PCIE_DevAttr_t));
    stDevAttr.enMode = (MI_PCIE_WorkMode_e)E_MI_PCIE_WORKMODE_RC;
    s32Ret = MI_PCIE_CreateDevice(devId, &stDevAttr);
    if (MI_SUCCESS != s32Ret)
    {
        return;
    }
    MI_PCIE_CreateDevice(devId);
}
  
```

➤ 相关主题

[MI_PCIE_DestroyDevice](#)

4.2. MI_PCIE_DestroyDevice

➤ 功能

反初始化 PCIe 设备。

➤ 语法

```
MI_S32 MI_PCIE_DestroyDevice(MI\_PCIE\_DEV devId);
```

➤ 形参

参数名称	参数含义	输入/输出
devId	设备 id。	输入

➤ 返回值

{

 [MI_SUCCESS](#) 成功。

 非 [MI_SUCCESS](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_pcie.h。
- 库文件：无

※ 注意

- 在 PCIe 设备上的通道已经创建的情况下去销毁 PCIe 设备，则 PCIe 的通道也会在内部销毁。

➤ 举例

参见 [MI_PCIE_CreateDevice](#) 举例。

➤ 相关主题

[MI_PCIE_CreateDevice](#)

4.3. MI_PCIE_CreateChannel

➤ 功能

创建 PCIe 的通道。

➤ 语法

```
MI_S32 MI_PCIE_CreateChannel(MI\_PCIE\_DEV devId, MI\_PCIE\_CHN chnId, MI\_PCIE\_ChnAttr\_t
*pstPcieChnAttr);
```

➤ 形参

参数名称	参数含义	输入/输出
devId	设备 id。	输入
chnId	通道 id。	输入
pstPcieChnAttr	通道属性。	输入

➤ 返回值

{ [MI_SUCCESS](#) 成功。
非 [MI_SUCCESS](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_pcie.h。
- 库文件：无

※ 注意

- 输入的设备 id 必须先创建。
- 同一设备上不能存在相同的通道 id。
- 不同的设备可以使用同一个通道 id。
- 仅能在物理连接的 PCIe 端口上才能建立 RX 与 TX 设备通道的连接。
- 若本地为 RX 设备，那么默认会连接远端 TX 设备上 id 为 “u32TargetChnId” 的通道，本地为 TX 设备时同理。
- 在 TX 和 RX 设备上建立连接的通道必须互相指定对方的 u32TargetChnId，否则出现内部错误，并在传输时打印错误。
- 远端所指定的 u32TargetChnId 与本地的通道号必须一致，否则会出现内部错误，并出现错误打印。
- 当设备处于 RC 模式时，使用的 PCIe 端口 0 上的设备时（设备 id 为 0 或者 1），u16TargetChipId 取值范围为[1, 128]，使用 PCIe 端口 1 上的设备时（设备 id 为 2 或者 3），u16TargetChipId 取值范围为[129, 255]，若不满足这种设定条件，则该 API 返回 [MI_ERR_PCIE_ILLEGAL_PARAM](#)。
- 当设备处于 EP 模式，u16TargetChipId 设定任何值都无任何意义，底层代码会根据 devId 判断其用的是那个 PCIe Port，从而得到 Target Id。
- 以上 u16TargetChipId 取值范围的限制请参考 [Target Id 定义](#)。

➤ 举例

```
typedef enum
{
    E_MI_PCIE_DEV0_RX,
    E_MI_PCIE_DEV0_TX,
    E_MI_PCIE_DEV1_RX,
    E_MI_PCIE_DEV1_TX,
    E_MI_PCIE_DEV_MAX
}MI_PCIE_DevType_e;

void SamplePcie(void)
{
    MI_PCIE_DevAttr_t stDevAttr;
    MI_PCIE_DEV devId = (MI_PCIE_DEV)E_MI_PCIE_DEV0_TX;
    MI_PCIE_CHN chnId = 0;
    MI_PCIE_ChnAttr_t stPcieChnAttr;
    MI_S32 s32Ret = 0;

    memset(&stDevAttr, 0, sizeof(MI_PCIE_DevAttr_t));
    stDevAttr.enMode = (MI_PCIE_WorkMode_e)E_MI_PCIE_WORKMODE_EP;
    s32Ret = MI_PCIE_CreateDevice(devId, &stDevAttr);
```

```

if (MI_SUCCESS != s32Ret)
{
    return;
}
memset(&stPcieChnAttr, 0, sizeof(MI_PCIE_ChAttr_t));
chnId = 8;
stPcieChnAttr.u16TargetChipId = 2;
stPcieChnAttr.u32TargetChnId = 5;
s32Ret = MI_PCIE_CreateChannel(devId, chnId, &stPcieChnAttr);
if (MI_SUCCESS != s32Ret)
{
    MI_PCIE_CreateDevice(devId);
    return;
}
MI_PCIE_DestroyChannel(devId, chnId);
MI_PCIE_DestroyDevice(devId);
}

```

➤ 相关主题

[MI_PCIE_DestroyChannel](#)

4.4. MI_PCIE_DestroyChannel

➤ 功能

销毁 PCIe 创建的通道。

➤ 语法

MI_S32 MI_PCIE_DestroyChannel([MI_PCIE_DEV](#) devId, [MI_PCIE_CHN](#) chnId);

➤ 形参

参数名称	参数含义	输入/输出
devId	设备 id。	输入
chnId	通道 id。	输入

➤ 返回值

{

[MI_SUCCESS](#) 成功。

 非 [MI_SUCCESS](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_pcie.h。
- 库文件：无

※ 注意

- 设备和通道必须已经创建。

➤ 举例

参见 [MI_PCIE_CreateChannel](#) 举例。

➤ 相关主题

[MI_PCIE_CreateChannel](#)

4.5. MI_PCIE_StartChannel

➤ 功能

使能通道数据流传输。

➤ 语法

MI_S32 MI_PCIE_StartChannel([MI_PCIE_DEV](#) devId, [MI_PCIE_CHN](#) chnId);

➤ 形参

参数名称	参数含义	输入/输出
devId	设备 id。	输入
chnId	通道 id。	输入

➤ 返回值

{

 [MI_SUCCESS](#) 成功。

 非 [MI_SUCCESS](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_pcie.h。
- 库文件：无

※ 注意

- 设备和通道必须依据创建。
- 通道开始输出传输后，MI SYS 才可以与之做 bind、inject、put 的操作。

➤ 举例

```
typedef enum
{
    E_MI_PCIE_DEV0_RX,
    E_MI_PCIE_DEV0_TX,
    E_MI_PCIE_DEV1_RX,
    E_MI_PCIE_DEV1_TX,
    E_MI_PCIE_DEV_MAX
}MI_PCIE_DevType_e;

void SamplePcie(void)
{
    MI_PCIE_DevAttr_t stDevAttr;
    MI_PCIE_DEV devId = (MI_PCIE_DEV)E_MI_PCIE_DEV0_TX;
    MI_PCIE_CHN chnId = 0;
    MI_PCIE_ChnAttr_t stPcieChnAttr;
```

```
MI_S32 s32Ret = 0;

memset(&stDevAttr, 0, sizeof(MI_PCIE_DevAttr_t));
stDevAttr.enMode = (MI_PCIE_WorkMode_e)E_MI_PCIE_WORKMODE_EP;
s32Ret = MI_PCIE_CreateDevice(devId, &stDevAttr);
if (MI_SUCCESS != s32Ret)
{
    return;
}
memset(&stPcieChnAttr, 0, sizeof(MI_PCIE_ChnAttr_t));
chnId = 8;
stPcieChnAttr.u16TargetChipId = 2;
stPcieChnAttr.u32TargetChnId = 5;
s32Ret = MI_PCIE_CreateChannel(devId, chnId, &stPcieChnAttr);
if (MI_SUCCESS != s32Ret)
{
    goto ERR0;
    return;
}
s32Ret = MI_PCIE_StartChannel(devId, chnId, &stPcieChnAttr);
if (MI_SUCCESS != s32Ret)
{
    goto ERR1;
    return;
}
MI_PCIE_StopChannel(devId, chnId);
ERR1:
MI_PCIE_DestroyChannel(devId, chnId);
ERR0:
MI_PCIE_DestroyDevice(devId);
}
```

- 相关主题
[MI_PCIE_StopChannel](#)

4.6. MI_PCIE_StopChannel

- 功能
停止通道的数据流传输。

- 语法
MI_S32 MI_PCIE_StopChannel([MI_PCIE_DEV](#) devId, [MI_PCIE_CHN](#) chnId);

- 形参

参数名称	参数含义	输入/输出
devId	设备 id。	输入
chnId	通道 id。	输入

- 返回值
 - [MI_SUCCESS](#) 成功。

非 [MI_SUCCESS](#) 失败，参照[返回值](#)。

➤ 依赖

- 头文件：mi_sys.h、mi_pcie.h。
- 库文件：无

※ 注意

- 设备和通道必须先创建，并且已经开始数据流传输。

➤ 举例

请参见 [MI_PCIE_StartChannel](#) 的举例。

➤ 相关主题

无。

4.7. MI_PCIE_DmaTask

➤ 功能

触发本地和远端 PCIe DMA 设备之间的数据传输任务。

➤ 语法

MI_S32 MI_PCIE_DmaTask([MI_PCIE_DEV](#) devId, [MI_PCIE_DmaTask_t](#) *pstDmaTask);

➤ 形参

参数名称	参数含义	输入/输出
devId	设备 id。	输入
pstDmaTask	配置 DMA 传输任务的数据结构体指针。	输入

➤ 返回值

{

 [MI_SUCCESS](#) 成功。

 非 [MI_SUCCESS](#) 失败，参照[返回值](#)。

 }

➤ 依赖

- 头文件：mi_sys.h、mi_pcie.h。
- 库文件：

※ 注意

- 设备必须先创建。
- 只能在 EP 端发起，可以参考 [PCIe 总线 DMA](#) 了解情况。
- 设备处于 RX 或者 TX 的模式下可以得出的数据传输方向，用户在发起 PCIe DMA 任务时，bReadFromRmt 的设定必须与当前设备的模式相匹配，否则会报错，这个标志位是为了防止用户做不合理的调用而做的二次检查。bReadFromRmt 表示的是是否从远端读取数据到本地，若为 TRUE 则本地必须处于 PCIe 的 RX 模式，若为 FALSE 则处于 TX 模式。若需要建立双向的 dma 传输，则需使用

RX 和 TX 两个设备，分别处理 TX/RX 模式的数据传输，这里的本地仅指 EP 端。

- u32BlockCount 必须小于等于 [MI_PCIE_DMA_BLOCK_MAX](#)。
- 结构体 MI_PCIE_DmaBlocks_t 中 phyLocalAddr 和 phyRemoteAddr 必须是连续的物理地址，并且在传输的前后，要考虑 CPU Cache 的问题。
- 在 EP 端 phyLocalAddr 代表的是本地的物理地址，phyRemoteAddr 代表的是远端的物理地址，在 EP 端的 APP 需要考虑如何获取远端的地址，例如通过 RpmMsg 与 RC 通讯来获取地址值。
- u32BlkSize 的单位为 byte，无对齐要求

➤ 举例

```
typedef enum
{
    E_MI_PCIE_DEV0_RX,
    E_MI_PCIE_DEV0_TX,
    E_MI_PCIE_DEV1_RX,
    E_MI_PCIE_DEV1_TX,
    E_MI_PCIE_DEV_MAX
}MI_PCIE_DevType_e;

void SamplePcie(void)
{
    MI_PCIE_DevAttr_t stDevAttr;
    MI_PCIE_DEV devId = (MI_PCIE_DEV)E_MI_PCIE_DEV0_RX;
    MI_PCIE_DmaTask_t stDmaTask;
    MI_S32 s32Ret = 0;

    memset(&stDevAttr, 0, sizeof(MI_PCIE_DevAttr_t));
    stDevAttr.enMode = (MI_PCIE_WorkMode_e)E_MI_PCIE_WORKMODE_EP;
    s32Ret = MI_PCIE_CreateDevice(devId, &stDevAttr);
    if (MI_SUCCESS != s32Ret)
    {
        return;
    }
    memset(&stDmaTask, 0, sizeof(MI_PCIE_DmaTask_t))
    stDmaTask.u32BlockCount = 1;
    stDmaTask.bReadFromRmt = TRUE;
    stDmaTask.stDmaBlocks[0].phyLocalAddr = 0xA8800000100000
    stDmaTask.stDmaBlocks[0].phyRemoteAddr = 0xA6600000100000
    stDmaTask.stDmaBlocks[0].u32BlkSize = 0x100000
    MI_PCIE_DmaTask(devId, &stDmaTask);
    MI_PCIE_DestroyDevice(devId);
}
```

➤ 相关主题

无

5. MI PCIE 数据类型

视频前处理相关数据类型、数据结构定义如下：

数据类型	定义
MI_PCIE_DMA_BLOCK_MAX	PCIe 最大 DMA 传输内存个数
MI_PCIE_DEV	PCIe 设备号
MI_PCIE_CHN	PCIe 通道号
MI_PCIE_WorkMode_e	PCIe 的工作模式
MI_PCIE_DevAttr_t	PCIe 设备属性
MI_PCIE_DmaBlocks_t	PCIe DMA 传输地址参数配置
MI_PCIE_DmaTask_t	PCIe DMA 传输任务配置
MI_PCIE_ChnAttr_t	PCIe 通道的属性

5.1. MI_PCIE_DMA_BLOCK_MAX

- 说明
PCIe 一次最大传输内存块个数。
- 定义
`#define MI_PCIE_DMA_BLOCK_MAX 8`
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
无

5.2. MI_PCIE_DEV

- 说明
PCIe 设备号定义。
- 定义
`typedef MI_S32 MI_PCIE_DEV;`
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
无

5.3. MI_PCIE_CHN

- 说明
PCIe 通道号定义。
- 定义
`typedef MI_S32 MI_PCIE_CHN;`
- 成员
无
- ※ 注意事项
无

- 相关数据类型及接口
无

5.4. MI_PCIE_WorkMode_e

- 说明
工作模式定义。

- 定义

```
typedef enum
{
    E_MI_PCIE_WORKMODE_RC,
    E_MI_PCIE_WORKMODE_EP
}MI_PCIE_WorkMode_e;
```

- 成员

成员名称	描述
E_MI_PCIE_WORKMODE_RC	PCIe Root Complex（主机）模式。
E_MI_PCIE_WORKMODE_EP	PCIe End Point（从机）模式。

- ※ 注意事项
无

- 相关数据类型及接口
无

5.5. MI_PCIE_DevAttr_t

- 说明
PCIe 设备参数。

- 定义

```
typedef struct MI_PCIE_DevAttr_s
{
    MI\_PCIE\_WorkMode\_e enMode;
}MI_PCIE_DevAttr_t;
```

- 成员

成员名称	描述
enMode	PCIe 设备工作模式。

- ※ 注意事项
无

- 相关数据类型及接口
无

5.6. MI_PCIE_DmaBlocks_t

- 说明
PCIe DMA 传输数据参数。
- 定义

```
typedef struct MI_PCIE_DmaBlocks_s
{
    MI_PHY phyLocalAddr;
    MI_PHY phyRemoteAddr;
    MI_U32 u32BlkSize;
}MI_PCIE_DmaBlocks_t;
```

- 成员

成员名称	描述
phyLocalAddr	本地物理地址。
phyRemoteAddr	远端物理地址
u32BlkSize	传输大小，单位 byte

- ※ 注意事项
无

- 相关数据类型及接口
MI_PCIE_DmaTask

5.7. MI_PCIE_DmaTask_t

- 说明
发起一次传输任务所需要的参数。
- 定义

```
typedef struct MI_PCIE_DmaTask_s
{
    MI_U32 u32BlockCount;
    MI_BOOL bReadFromRmt;
    MI\_PCIE\_DmaBlocks\_t stDmaBlocks[MI\_PCIE\_DMA\_BLOCK\_MAX];
}MI_PCIE_DmaTask_t;
```

- 成员

成员名称	描述
u32BlockCount	本次传输的数据块个数。
bReadFromRmt	本次传输的方向。

stDmaBlocks	数据块数据组成的数组
-------------	------------

※ 注意事项
无

➤ 相关数据类型及接口
MI_PCIE_DmaTask

5.8. MI_PCIE_ChAttr_t

➤ 说明
PCIe 通道属性的结构体。

➤ 定义

```
typedef struct MI_PCIE_ChAttr_s
{
    MI_U16 u16TargetChipId;
    MI_U32 u32TargetChnId;
}MI_PCIE_ChAttr_t;
```

➤ 成员

成员名称	描述
u16TargetChipId	需要传输的远端的 Target Id
u32TargetChnId	远端 Chip 上对应的 PCIe 的通道 ID

※ 注意事项
无

5.8.1 Target Id 定义

以两个 PCIe Port 口的 SOC 举例，从 RC 端的角度看，每个 EP 端的 SOC 都必须有唯一的 Target Id，反之在 EP 的角度看 RC，也必须成立。

5.8.1.1. RC 模式：

在 RC 的模式下，若 PCIe 的连接不带 SWITCH，则一个 RC 最多连接两个 EP。这种情况下 Target Id 是根据 RC 的 PCIe Port 口使用的情况来区分，在 Port0 上与之连接的 EP 的 Target Id 为 1，在 Port1 则为 129。

另外一种情况，如下图中 PCIe Port0 和 Port1 上都接了 SWITCH，在每个 port 上的 RC 设备内部和 SWITCH 内部会使用 PCIe BUS0、BUS1 和 BUS2，则从 BUS3 开始 Port0 连接 EP 端的 SOC B/C/D，Port1 连接 EP 端的 SOC E/F/G。

在 Port0 上使用 SWITCH 的 BUS ID 就是 Target Id，而在 Port1 上 Target Id = BUS ID + 128。下图中 SOC B/C/D 的 Target Id 为 3/4/5，SOC E/F/G 的 Target Id 为 131/132/133。

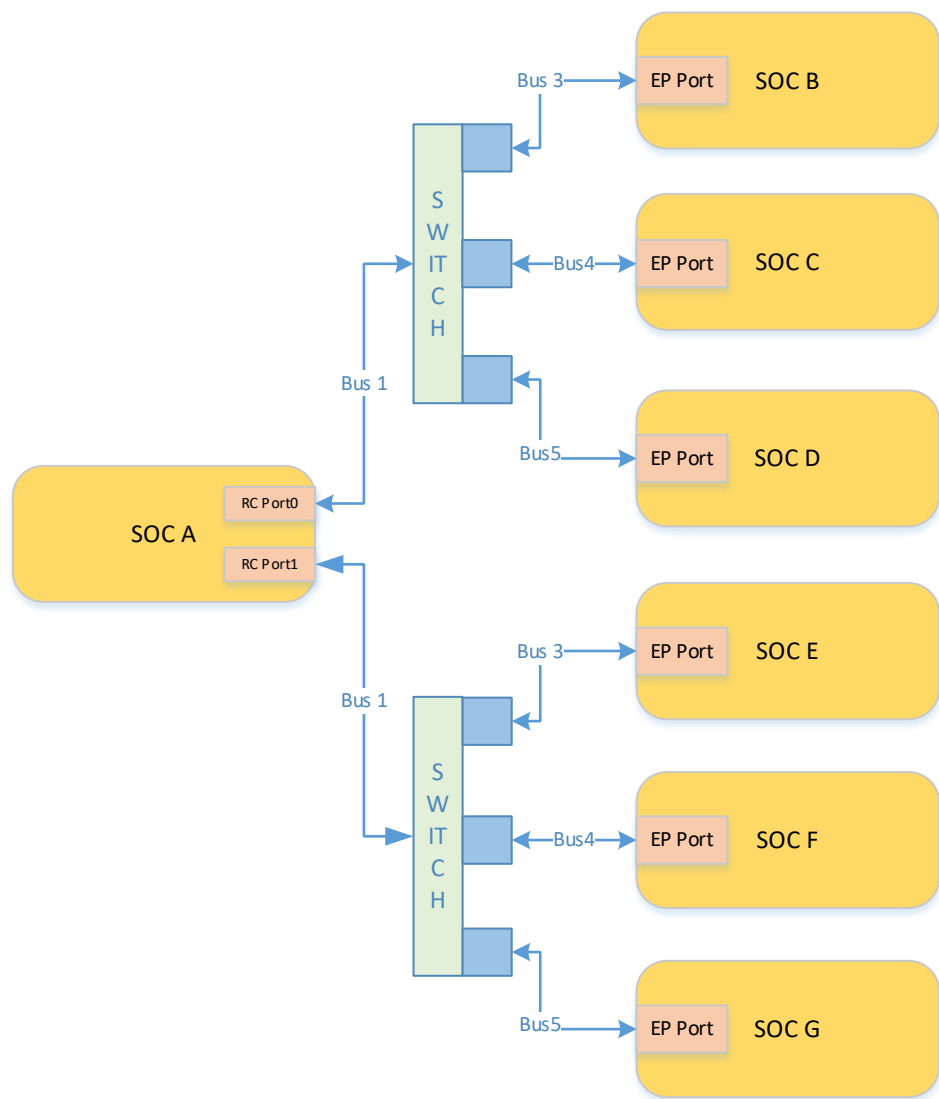
	SOC B	SOC C	SOC D	SOC E	SOC F	SOC G
Port Id	0	0	0	1	1	1
Bus Id	3	4	5	3	4	5

	SOC B	SOC C	SOC D	SOC E	SOC F	SOC G
Target Id	3	4	5	131	132	133

SWITCH 的 BUS ID 和 RC 端 SOC 的 PCIe Port 口在物理结构上都是对应不变的，因此可通过唯一的 ID 能够在板子上找到指定的 EP 端的 SOC。

另外一种情况，例如有的 PCIe Port 口接入 SWITCH，有的未接入，则未接入 SWITCH 的 SOC 所对应的 Target Id 取决于连接的是哪个 Port 口，为 1 或者 128，而连接 SWITCH 的 Port 按照上述已知方法来确定 Target Id。

目前 Target Id 映射规则仅支持最大两个 PCIe Port。



5.8.1.2. EP 模式:

在 EP 端的 Pcie Port 有且只有一个 RC 与之连接，那么 EP 端发起通讯的 Target Id 为 PCIe 的 Port Id，为 0 或者 1。

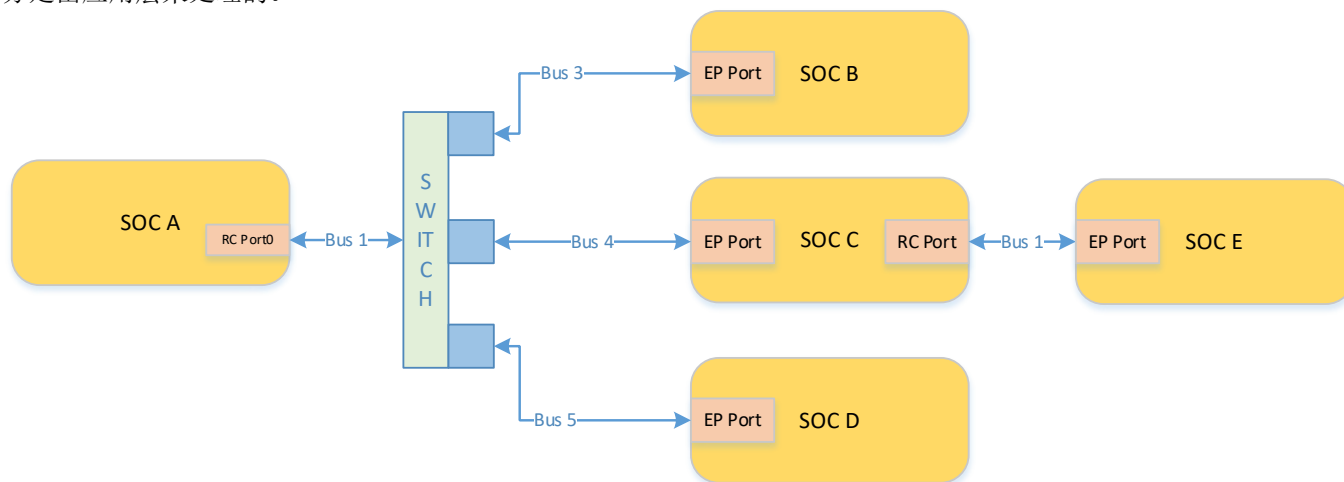
5.8.1.3. 混合 RC/EP 模式:

PCIe 的两个 Port 一个作为 RC，另一个作为 EP。这种属于二级级联的场景。那么在 RC 的 Port 上与之连接的 SOC Targ

et Id 适用于以上 RC 模式的规则，同理 EP Port 上连接的 SOC Target Id 适用于 EP 模式。

如下图所示，SOC C 的两个 PCIe Port 分别处于 EP 模式和 RC 模式，则在 SOC C 上，假设与 SOC A 通讯的 PCIe Port i d 为 0，则 Target Id = 0，模式是 EP 模式，与 SOC E 通讯，Target Id = 129，RC 模式。

在二级级联的场景下，SOC A 若要与 SOC E 进行双向的通讯，则需要在 SOC C 上做一次转发（Bridge 功能），这部分是由应用层来处理的。



- 相关数据类型及接口
无

6. MI PCIE 返回值

MI_PCIE API 返回值如下表所示。

表 6-1 MI_PCIE API 返回值

错误码	宏定义	描述
0xA0252001	MI_ERR_PCIE_INVALID_DEVID	使用错误的设备 id
0xA0252002	MI_ERR_PCIE_INVALID_CHNID	使用错误的通道 id
0xA0252003	MI_ERR_PCIE_ILLEGAL_PARAM	使用错误的参数
0xA0252004	MI_ERR_PCIE_EXIST	通道已经存在
0xA0252005	MI_ERR_PCIE_UNEXIST	通道不存在
0xA0252006	MI_ERR_PCIE_NULL_PTR	空指针
0xA0252007	MI_ERR_PCIE_NOT_CONFIG	未配置
0xA0252008	MI_ERR_PCIE_NOT_SUPPORT	不支持当前配置
0xA0252009	MI_ERR_PCIE_NOT_PERM	不允许当前配置
0xA025200C	MI_ERR_PCIE_NOMEM	无系统内存
0xA025200D	MI_ERR_PCIE_NOBUF	无连续物理内存
0xA025200E	MI_ERR_PCIE_BUF_EMPTY	物理内存池为空
0xA025200F	MI_ERR_PCIE_BUF_FULL	物理内存池满
0xA0252011	MI_ERR_PCIE_BADADDR	错误的地址
0xA0252012	MI_ERR_PCIE_BUSY	资源冲突
0xA0252010	MI_ERR_PCIE_NOTREADY	设备当前不可用
0xA0252013	MI_ERR_PCIE_CHN_NOT_STARTED	通道未开始
0xA0252014	MI_ERR_PCIE_CHN_NOT_STOPPED	通道未结束
0xA025201F	MI_ERR_PCIE_UNDEFINED	未知错误

7. MI_PCIE 调试信息

7.1. cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_pcie/mi_pciex
```

其中 x 为 pcie 的设备 id。

Pipeline 部分请参考 MI_SYS 调试信息。

RC 设备所包含的通道信息:

-----Chn[0]-----							
TargetChnId	LinkDev	TargetId	bBreakPipe	bRxMultiTask	MsgRxCnt	MsgTxCnt	
0	0xffffffff8002cf7280	1	0	0	1300	646	
Mode:RC							
PrepareRmtBuf	SendRmtBuf	SendRmtFail	DmaDone	DmaFail			
655	646	0	645	0			
TaskId	Status	PreInCnt	OnPollCnt	EqCnt	DropCnt	FinishCnt	CheckCnt
0	pending	0	65	65	0	65	2496
1	pending	0	65	65	0	65	2517
2	pending	0	65	65	0	65	2511
3	pending	0	65	65	0	65	2518
4	pending	0	65	65	0	65	2504
5	running	0	65	65	0	64	2504
6	pending	0	64	64	0	64	2456
7	pending	0	64	64	0	64	2501
8	pending	0	64	64	0	64	2499
9	pending	0	64	64	0	64	2487
ERR Message Info:							
R-ChnIdErr	R-ChnBufErr	R-ChnBusyErr	R-UserExit	L-ChnBufErr	L-ChnBusyErr	L-UserExit	
0	0	0	0	0	0	0	

EP 设备所包含的通道信息:

-----Chn[0]-----							
TargetChnId	LinkDev	TargetId	bBreakPipe	bRxMultiTask	MsgRxCnt	MsgTxCnt	
0	0xffffffff8004d69180	1	0	0	10944	21895	
Mode:EP							
GrabRmtBuf	GrabRmtFail	RecvBuf	TriggerDma	DmaDone	DmaFail		
10953	0	0	10943	10942	0		
TaskId	Status	PreInCnt	OnPollCnt	EqCnt	DropCnt	FinishCnt	CheckCnt
0	running	1096	0	1096	0	1095	178111
1	running	1097	0	1097	0	1095	178370
2	running	1095	0	1095	0	1094	178180
3	running	1095	0	1095	0	1094	178171
4	running	1095	0	1095	0	1094	178336
5	running	1095	0	1095	0	1094	178289
6	running	1095	0	1095	0	1094	178358
7	running	1095	0	1095	0	1094	178198
8	running	1095	0	1095	0	1094	178268
9	running	1095	0	1095	0	1094	178267
ERR Message Info:							
R-ChnIdErr	R-ChnBufErr	R-ChnBusyErr	R-UserExit	L-ChnBufErr	L-ChnBusyErr	L-UserExit	
0	0	1	0	0	0	0	

➤ 参数说明

参数			描述
Channel State	状态栏	Chn[id]	Id 表示通道号
		TargetChnId	远端的通道号
		LinkDev	内部 Link 设备地址
		TargetId	当前通道的 Target Id
		bBreakPipe	是否阻断内部 pipeline 数据流的标志
		bRxMultiTask	RX 设备是否使用多 Task 的标志
		MsgRxCnt	内部 PCIe 消息发送次数
		MsgTxCnt	内部 PCIe 消息接收次数
		Mode	表示当前的 channel 处于哪种 PCIe 的传输模式下，EP/RC
	RC 设备	PrepareRmtBuf	处于 RX 模式时才会累加，收到 EP TX 设备申请内存的请求次数
		SendRmtBuf	内部的 Task 状态机，向 EP 请求 Dma 的次数
		SendRmtFail	内部的 Task 状态机，向 EP 请求 Dma 出错的次数
		DmaDone	内部的 Task 状态机，收到 EP 反馈的 Dma 成功次数
		DmaFail	内部的 Task 状态机，收到 EP 反馈的 Dma 失败次数
	EP 设备	GrabRmtBuf	处于 TX 模式才会累加，请求 RC 申请内存的次数
		GrabRmtFail	内部的 Task 状态机，申请内存请求返回失败的次数
		RecvBuf	内部的 Task 状态机，收到 RC 申请的内存成功的次数
		TriggerDma	内部的 Task 状态机，驱动 Dma 的请求次数
		DmaDone	内部的 Task 状态机，驱动 Dma 成功的次数
		DmaFail	内部的 Task 状态机，驱动 Dma 失败的次数
	Task Infomation	TaskId	内部 Task 的 Id 号
		Status	内部 Task 实时的状态，会有多个，状态分为：“unused”（未使用）、“pending”（挂起）、“used”（已使用）、“running”（正在运行）、“done”（完成）、“retry”（重试）、“drop”（放弃）
		PreInCnt	TX 设备上内部 Task 输入准备次数。
		OnPollCnt	RX 设备上内部 Task 请求内存次数
		EqCnt	内部 Task 输入或者输出的内存获取次数
		DropCnt	因为异常放弃内部 Task 的次数
		FinishCnt	内部 Task 完成的次数
		CheckCnt	内部 Task 检查状态的次数
	ERR Message Info	R-ChnIdErr	远端的 Channel Id 对应的通道没有被创建，统计出错次数
		R-ChnBufErr	远端通道无法申请内存的次数

参数		描述
	R-ChnBusyErr	远端通道的工作线程十分忙碌，导致无法响应请求的次数
	R-UserExit	远端通道被用户主动退出的次数
	L-ChnBufErr	本地通道无法申请内存
	L-ChnBusyErr	本地通道的工作线程十分忙碌，导致无法响应请求的次数
	L-UserExit	本地通道被用户主动退出的次数

7.2. echo

➤ 调试信息

```
# echo dumpLinkDev 1 >/proc/mi_modules/mi_pcie/mi_pcie2
```

```
*****Start dump port 1 link device info*****
-----Instance 0xffffffff8003f9d680, Target id 129, RxDev 2 TxDev 3-
RefCnt      HalIns      RxCnt      TxCnt
  2 0xffffffff8002183e00  896148    448074
*****End dump port 1 link device info*****
```

功能	获取内部 RPMSG 设备信息	
命令	echo dumpLinkDev [PciePort] >/proc/mi_modules/mi_pcie/mi_pciex	
参数说明	[PciePort]: Pcie 的端口号	
举例	echo dumpLinkDev 1 >/proc/mi_modules/mi_pcie/mi_pcie2	
调试信息说明	Instance	内部资源地址
	Target id	RPMSG 链接通讯的 Target Id
	Rx Dev	RX 模式的设备号
	Tx Dev	TX 模式的设备号
	RefCnt	设备被引用的次数
	HalIns	底层 RPMSG 设备节点地址
	RxCnt	接收消息的次数
	TxCnt	发送消息的次数

➤ 调试信息

```
# echo clearLinkMsgCnt 1 129 >/proc/mi_modules/mi_pcie/mi_pcie2
```

功能	清除内部 RPMSG 收发计数
命令	echo clearLinkMsgCnt [PciePort] [TargetId] > /proc/mi_modules/mi_pcie/mi_pciex
参数说明	[PciePort]: Pcie 的端口号, [TargetId]: PCIe Target Id 号

举例	echo clearLinkMsgCnt 1 129 >/proc/mi_modules/mi_pcie/mi_pcie2
----	---

➤ 调试信息

echo clearChnMsgCnt 0 >/proc/mi_modules/mi_pcie/mi_pcie2

功能	清除通道内部的所有消息计数
命令	echo clearChnMsgCnt [ChnId] > /proc/mi_modules/mi_pcie/mi_pcie2
参数说明	[ChnId]: PCIe 设备所对应的通道号
举例	echo clearChnMsgCnt 0 >/proc/mi_modules/mi_pcie/mi_pcie2
调试信息说明	该命令会清零调试计数，在某些问题即将发送之前，清零计数有利于还原当前内部真实状态，会清除的计数如下： Channel State 中的 RC 设备、EP 设备、ERR Message Info 所有计数信息。

➤ 调试信息

echo breakPipe 0 1 >/proc/mi_modules/mi_pcie/mi_pcie2

功能	设置是否断开内部 pipeline 的数据流
命令	echo breakPipe [chnId] [OnOff] > /proc/mi_modules/mi_pcie/mi_pcie2
参数说明	[ChnId]: PCIe 设备所对应的通道号 [OnOff]: 该功能开关，用 0 和 1 表示，0 表示关，1 表示开
举例	echo breakPipe 0 1 >/proc/mi_modules/mi_pcie/mi_pcie2
调试信息说明	如果打开该功能，则会停止处理内部数据流。UserExit 的异常计数会一直增加。

➤ 调试信息

echo rxMultiTask 0 1 >/proc/mi_modules/mi_pcie/mi_pcie2

功能	设置是否打开 RX 设备上多 task 处理机制
命令	echo rxMultiTask [chnId] [OnOff] > /proc/mi_modules/mi_pcie/mi_pcie2
参数说明	[ChnId]: PCIe 设备所对应的通道号 [OnOff]: 该功能开关，用 0 和 1 表示，0 表示关，1 表示开
举例	echo rxMultiTask 0 1 >/proc/mi_modules/mi_pcie/mi_pcie2
调试信息说明	实验性功能，功能打开后 RX 设备会同时处理多个 Task，在某些情况下可能会增加数据传输的速度，但是会增加 cpu loading。

8. RPMSG IOCTL 参考

IOCTL 命令	功能
SS_RPMSG_CREATE_EPT_IOCTL	创建 RPMsg Endpoint 设备节点
SS_RPMSG_DESTROY_EPT_IOCTL	销毁 RPMsg Endpoint 设备节点
SS_RPMSG_DEVICES_INFO_IOCTL	获取当前连接上的 SOC 列表
SS_RPMSG_QUERY_IOCTL	查询指定从片的状态信息
SS_RPMSG_LOAD_IMAGE_IOCTL	下载 image 到从片的指定地址
SS_RPMSG_BOOTUP_IOCTL	通知从片从指定地址开始启动
SS_RPMSG_BOOTUP_DONE_IOCTL	通知主片本机已经启动完成(APP 已经 ready)

8.1. SS_RPMSG_CREATE_EPT_IOCTL

➤ 功能

创建和 target chip 通信的 RPMsg Endpoint 设备节点

➤ 语法

```
ioctl(int fd, SS_RPMSG_CREATE_EPT_IOCTL, struct ss_rpmsg_endpoint_info *info);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsg_ctrl0 的文件句柄。	输入
info	RPMsg Endpoint 的属性。	输入/输出

➤ 返回值

返回值 { 0 表示成功。
-1 表示失败，查看 errno 获取原因。

➤ 依赖

- 头文件: drivers/ssstar/include/ssstar_rpmsg.h
errno.h
- 库文件

※ 注意

- 主片(RC)只能创建和从片(EP)通信的 RPMsg Endpoint。
- 从片(EP)只能创建和主片(RC)通信的 RPMsg Endpoint。
- 每次 ioctl 都会新建一个/dev/rpmsgX 节点(如:rpmsg0)，表示一个虚拟的通信通道。

- 每个 RC 口或 EP 口长出来的 RPMsg bus，其地址空间是独立的，不同 mode&target_id 组合表示一个独立的 RPMsg bus，所以不同 mode&target_id 组合可以使用一样的 src address。
- 可以使用完全相同的 [struct ss_rpmsg_endpoint_info](#) 多次调用，创建多个 /dev/rpmsgX 节点，但是当这些节点中的一个被 open 后，其他都会 open 失败，同一时间只会允许一个处于 open 状态。
- /dev/rpmsgX 里面的 X 由 [struct ss_rpmsg_endpoint_info](#) 里面的 id 返回。
- 如使用 mdev，则 ioctl 返回后，/dev/rpmsgX 节点可能会还没有被 mdev 建立，会有延迟。
- 在合法的 src&dst address 空间内可以创建任意多个 /dev/rpmsgX 节点。

➤ 举例

■ 主片(RC)端 demo

```

struct ss_rpmsg_endpoint_info info;
int fd, eptFd;
char eptPath[256];
char data[512];

memset(&info, 0, sizeof(info));
info.src = EPT_ADDR_MACRO(EPT_TYPE_CUSTOMER, 1);
info.dst = EPT_ADDR_MACRO(EPT_TYPE_CUSTOMER, 2);
snprintf(info.name, sizeof(info.name), "demo");
info.mode = RPMSG_MODE_RC_PORT;
info.target_id = 1;

fd = open("/dev/rpmsg_ctrl0", O_RDWR);
if (fd < 0)
    return -1;
if (ioctl(fd, SS_RPMSG_CREATE_EPT_IOCTL, &info) < 0) {
    close(fd);
    return -1;
}

snprintf(eptPath, sizeof(eptPath), "/dev/rpmsg%d", info.ept_id);
eptFd = open(eptPath, O_RDWR);
if (eptFd < 0)
    return -1;
...
read(eptFd, data, sizeof(data));
write(eptFd, data, size);
...
ioctl(eptFd, SS_RPMSG_DESTROY_EPT_IOCTL);
close(eptFd);

```

■ 从片(EP)端 demo

```

struct ss_rpmsg_endpoint_info info;

```

```
int fd, eptFd;
char eptPath[256];
char data[512];

memset(&info, 0, sizeof(info));
info.src = EPT_ADDR_MACRO(EPT_TYPE_CUSTOMER, 2);
info.dst = EPT_ADDR_MACRO(EPT_TYPE_CUSTOMER, 1);
snprintf(info.name, sizeof(info.name), "demo");
info.mode = RPMSG_MODE_EP_PORT;
info.target_id = 1;

fd = open("/dev/rpmsg_ctrl0", O_RDWR);
if (fd < 0)
    return -1;
if (ioctl(fd, SS_RPMSG_CREATE_EPT_IOCTL, &info) < 0) {
    close(fd);
    return -1;
}

snprintf(eptPath, sizeof(eptPath), "/dev/rpmsg%d", info.ept_id);
eptFd = open(eptPath, O_RDWR);
if (eptFd < 0)
    return -1;
...
read(eptFd, data, sizeof(data));
write(eptFd, data, size);
...
ioctl(eptFd, SS_RPMSG_DESTROY_EPT_IOCTL);
close(eptFd);
```

➤ 相关主题

[SS_RPMSG_DESTROY_EPT_IOCTL](#)

8.2. SS_RPMSG_DESTROY_EPT_IOCTL

➤ 功能

销毁掉对应的 RPMsg Endpoint 设备节点。

➤ 语法

```
ioctl(int fd, SS_RPMSG_DESTROY_EPT_IOCTL);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsgX(如 rpmsg0)的文件句柄。	输入

--	--	--

➤ 返回值

返回值 { 0 表示成功。
-1 表示失败，查看 `errno` 获取原因。

➤ 依赖

- 头文件： `drivers/ssstar/include/ssstar_rpmsg.h`
`errno.h`
- 库文件

※ 注意

- 创建 RPMsg Endpoint(/dev/rpmsgX)使用的是/dev/rpmsg_ctrl0 文件句柄，而销毁 /dev/rpmsgX 节点的 `ioctl` 使用的是/dev/rpmsgX 文件句柄，在调用 `SS_RPMSG_DESTROY_EPT_IOCTL` 后，对应的 RPMsg Endpoint 设备节点会在其最后一个文件句柄被 `close` 后销毁。
- RPMsg Endpoint 的创建/销毁和 `open/close` 是独立的，`open` 后开始 `read/write` 使用，`close` 后，可以重新 `open` 再次使用(`read/write`)，如无其他需求，不必每次都重新创建。当不再需要了，可以调用 `SS_RPMSG_DESTROY_EPT_IOCTL` 进行销毁。

➤ 举例

参见 [SS_RPMSG_CREATE_EPT_IOCTL](#) 举例。

➤ 相关主题

[SS_RPMSG_CREATE_EPT_IOCTL](#)

8.3. SS_RPMSG_DEVICES_INFO_IOCTL

➤ 功能

查询当前 PCIe ports 上连接的 chips 列表，返回设备个数和 `target_id` 列表。

➤ 语法

```
int ioctl(int fd, SS_RPMSG_DEVICES_INFO_IOCTL, struct rpmsg_devices_info *info);
```

➤ 形参

参数名称	参数含义	输入/输出
<code>fd</code>	/dev/rpmsg_ctrl0 的文件句柄。	输入
<code>info</code>	指定查询的设备类型，并提供存储返回值的 <code>buffer</code> 。	输入/输出

➤ 返回值

返回值 { 0 表示成功。
-1 表示失败，查看 `errno` 获取原因

➤ 依赖

- 头文件: `drivers/ssstar/include/ssstar_rpmsg.h`
`errno.h`
- 库文件

※ 注意

- 主片(RC)/从片(EP)都可以使用这个接口进行查询, 主片查询到的是接在其 RC 口上从片的 `target id` 列表。从片则查询到的是有连接上主片的 EP 口 `id` 列表。

➤ 举例(RC 端)

```
struct rpmsg_devices_info info;
unsigned short *pTargetIDs;

info.mode = RPMSG_MODE_RC_PORT;
info.count = 100;
info.buffer = (unsigned long long)malloc(100 * sizeof(unsigned int));

fd = open("/dev/rpmsg_ctrl0", O_RDWR);
if (fd < 0)
    return -1;

if (ioctl(fd, SS_RPMSG_DEVICES_INFO_IOCTL, &info) < 0) {
    close(fd);
    return -1;
}
printf("Current %d connected slaves of RC_PORT mode: \n", info.count);
pTargetIDs = (unsigned short *)info.buffer;
for (int i = 0; i < info.count; i++)
    printf("target_id %hu\n", pTargetIDs[i]);
free(info.buffer);
close(fd);
```

➤ 相关主题

无

8.4. SS_RPMSG_QUERY_IOCTL

➤ 功能

查询指定 `device` 的信息。

➤ 语法

```
int ioctl(int fd, SS_RPMSG_QUERY_IOCTL, struct ss_rpmsg_query_info *info);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsg_ctrl0 的文件句柄。	输入
info	指定查询的信息。	输入/输出

➤ 返回值

返回值 {
0 操作成功。
-1 表示失败，查看 `errno` 获取原因

➤ 依赖

- 头文件: `drivers/ssstar/include/ssstar_rpmsg.h`
`errno.h`
- 库文件

※ 注意

- [struct ss_rpmsg_query_info->mode](#) 只能为 `RPMSG_MODE_RC_PORT`。
主片(RC)端，用这个接口，查询从片处于哪个阶段，如果是 [RPMSG_STAGE_RTOS](#) 则表示这个从片还没有被主片引导，主片需要对这个从片进行引导启动操作(使用 [SS_RPMSG_LOAD_IMAGE_IOCTL](#) 下载 image，然后使用 [SS_RPMSG_BOOTUP_IOCTL](#) 发起 bootup)。

➤ 举例

```
struct ss_rpmsg_query_info info;

info.mode = RPMSG_MODE_RC_PORT;
info.target_id = 1;
info.param = RPMSG_QUERY_STAGE;

fd = open("/dev/rpmsg_ctrl0", O_RDWR);
if (fd < 0)
    return -1;

if (ioctl(fd, SS_RPMSG_QUERY_IOCTL, &info) < 0) {
    close(fd);
    fprintf(stderr, "ioctl SS_RPMSG_QUERY_IOCTL error:%s", strerror(errno));
    return -1;
}
printf("Current stage of slave device %u:%u: is %llu \n", info.mode, info.target_id, info.value);
close(fd);
```

➤ 相关主题

无

8.5. SS_RPMSG_LOAD_IMAGE_IOCTL

➤ 功能

下载 image 到指定 device 的指定地址上。

➤ 语法

```
int ioctl(int fd, SS_RPMSG_LOAD_IMAGE_IOCTL, struct ss_rpmsg_image_info *image_info);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsg_ctrl0 的文件句柄。	输入
image_info	指定 image buffer 物理地址，加载地址等信息。	输入

➤ 返回值

返回值 { 0 操作成功。
-1 表示失败，查看 errno 获取原因

➤ 依赖

- 头文件： drivers/sstar/include/sstar_rpmsg.h
errno.h
- 库文件

※ 注意

- 无

➤ 举例

```
struct ss_rpmsg_image_info info;
FILE *fp;
int fd;
int filelen;
MI_PHY phyAddr;
void *virtAddr;

info.mode = RPMSG_MODE_RC_PORT;
info.target_id = 1;
snprintf(info.name, sizeof(info.name), "kernel");

fp = fopen("kernel", "r");
if (fp == NULL)
    return -1;

fseek(fp, 0, SEEK_END);
filelen = ftell(fp);
fseek(fp, 0, SEEK_SET);
```

```

if (MI_SUCCESS == MI_SYS_MMA_Alloc(0, NULL, filelen, &phyAddr)) {
    if (MI_SUCCESS == MI_SYS_Mmap(phyAddr, filelen, (void *)&virtAddr, TRUE)) {
        fread(virtAddr, filelen, 1, fp);
        fclose(fp);
        MI_SYS_FlushInvCache(virtAddr, filelen);
        MI_SYS_Munmap(virtAddr, filelen);
    } else {
        MI_SYS_MMA_Free(0, phyAddr);
        fclose(fp);
        return -1;
    }
} else {
    fclose(fp);
    Return -1;
}
fclose(fp);
info.addr_phys = phyAddr;
info.load_addr_phys = 0x00008000;

fd = open("/dev/rpmsg_ctrl0", O_RDWR);
if (fd < 0) {
    MI_SYS_MMA_Free(0, phyAddr);
    fclose(imageFP);
    return -1;
}

if (ioctl(fd, SS_RPMSG_LOAD_IMAGE_IOCTL, &info) < 0) {
    close(fd);
    MI_SYS_MMA_Free(0, phyAddr);
    fprintf(stderr, "ioctl SS_RPMSG_LOAD_IMAGE_IOCTL error:%s", strerror(errno));
    return -1;
}
close(fd);
MI_SYS_MMA_Free(0, phyAddr);
return 0;

```

➤ 相关主题

无

8.6. SS_RPMSG_BOOTUP_IOCTL

➤ 功能

通知 target chip 从指定的地址开始后续启动。

➤ 语法

```
int ioctl(int fd, SS_RPMSG_BOOTUP_IOCTL, struct ss_rpmsg_bootup *info);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsg_ctrl0 的文件句柄。	输入
info	指定启动入口地址和 bootargs 字符串等信息。	输入

➤ 返回值

{
 0 操作成功。
 -1 表示失败，查看 errno 获取原因

➤ 依赖

- 头文件： drivers/sstar/include/sstar_rpmsg.h
errno.h
- 库文件

※ 注意

- [ss_rpmsg_bootup_info](#)->mode 只能是 [RPMSG_MODE_RC_PORT](#)。
主片(RC)端在使用 [SS_RPMSG_LOAD_IMAGE_IOCTL](#) 完成 image 下载后，调用这个接口通知从片开始启动。

➤ 举例

```

struct ss_rpmsg_bootup_info info;

info.mode = RPMSG_MODE_RC_PORT;
info.target_id = 1;
info.flags = RPMSG_BOOTUP_FLAG_WAIT_APP;
info.entry = 0x2008000;
info.bootargs = "init=/linuxrc LX_MEM=0x7FE0000
mma_heap=mma_heap_name0,miu=0,sz=0x1E00000 cma=2M
mmap_reserved=fb,miu=0,sz=0x300000,max_start_off=0x7C00000,max_end_off=0x7F00000";

fd = open("/dev/rpmsg_ctrl0", O_RDWR);
if (fd < 0)
    return -1;

if (ioctl(fd, SS_RPMSG_BOOTUP_IOCTL, &info) < 0) {
    close(fd);
    fprintf(stderr, "ioctl SS_RPMSG_BOOTUP_IOCTL error:%s", strerror(errno));
    return -1;
}
close(fd);

```

➤ 相关主题

[SS_RPMSG_LOAD_IMAGE_IOCTL](#)

8.7. SS_RPMSG_BOOTUP_DONE_IOCTL

➤ 功能

通知主片(RC)，该从片已经 bootup 完成，从片上的 APP 已经准备好。

➤ 语法

```
int ioctl(int fd, SS_RPMSG_BOOTUP_DONE_IOCTL, struct ss_rpmsg_bootup_done *info);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsg_ctrl0 的文件句柄。	输入
info	指定从哪个 EP 模式的 PCIe 发出通知，这也决定了通知哪个主片。	输入

➤ 返回值

返回值 {
0 操作成功。
-1 表示失败，查看 errno 获取原因

➤ 依赖

- 头文件： drivers/sstar/include/sstar_rpmsg.h
errno.h
- 库文件

※ 注意

- [struct ss_rpmsg_bootup_done](#)->mode 只能为 RPMSG_MODE_EP_PORT。
当从片上的 APP 准备好了，可以使用这个接口通知主片其已经 ready，可以接收主片发过来的信息了。

➤ 举例

```
struct ss_rpmsg_bootup_done info;

info.mode = RPMSG_MODE_EP_PORT;
info.target_id = 1;

fd = open("/dev/rpmsg_ctrl0", O_RDWR);
if (fd < 0)
    return -1;

if (ioctl(fd, SS_RPMSG_BOOTUP_DONE_IOCTL, & info) < 0) {
    close(fd);
    fprintf(stderr, "ioctl SS_RPMSG_BOOTUP_DONE_IOCTL error:%s", strerror(errno));
    return -1;
}
```

```
close(fd);
```

➤ 相关主题

[SS_RPMSG_BOOTUP_IOCTL](#)

9. RPMSG API 参考

API 名	功能
read	接收数据
write	发送数据

9.1. read

➤ 功能

接收数据。

➤ 语法

```
ssize_t read(int fd, void *buf, size_t count);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsgX(如:rpmsg0)的文件句柄。	输入
buf	buffer 地址, 用于存放读取到的数据	输出
count	buffer 的 size	输入

➤ 返回值

返回值 {
 >= 0 读取到的 bytes 个数。
 -1 失败, 检查 errno 获取原因。

➤ 依赖

- 头文件: `unistd.h`
 `errno.h`
- 库文件

※ 注意

- 支持使用 Linux select/poll/epoll 接口监视/dev/rpmsgX 的文件句柄(fd), 检查是有待读取的数据。

➤ 举例

参见 [SS_RPMSG_CREATE_EPT_IOCTL](#) 举例。

➤ 相关主题

[SS_RPMSG_CREATE_EPT_IOCTL](#)

9.2. write

➤ 功能

发送数据。

➤ 语法

```
ssize_t write(int fd, const void *buf, size_t count);
```

➤ 形参

参数名称	参数含义	输入/输出
fd	/dev/rpmsgX(如:rpmsg0)的文件句柄。	输入
buf	buffer 地址, 存放要发送的数据。	输入
count	待发送数据的字节数。	输入

➤ 返回值

返回值 {
 >= 0 已发送的字节数。
 -1 失败, 检查 errno 获取原因。

➤ 依赖

- 头文件: unistd.h
 errno.h
- 库文件

※ 注意

- 一次 write, 会使用一个 RPMsg 数据包发送, 最多携带 496 个 bytes, 所以需要检查返回值, 当返回值不是-1 时, 并且小于 count 时, 可以多次调用 write 来完成剩下数据的发送。

➤ 举例

参见 [SS_RPMMSG_CREATE_EPT_IOCTL](#) 举例。

➤ 相关主题

[SS_RPMMSG_CREATE_EPT_IOCTL](#)

10. RPMSG 接口数据类型

视频前处理相关数据类型、数据结构定义如下。

数据类型	定义
RPMSG_MODE_RC_PORT	表示 target 所连接的 RPMsg bus 是长在本机 RC 口上的
RPMSG_MODE_EP_PORT	表示 target 所连接的 RPMsg bus 是长在本机 EP 口上的
RPMSG_MODE_UNKNOWN	表示未知 RPMsg bus 模式
EPT_TYPE_SIGMASTAR	表示非用户使用的(Sigmastar 使用)RPMsg Endpoint 地址
EPT_TYPE_CUSTOMER	表示用户使用的(非 Sigmastar 使用)RPMsg Endpoint 地址
EPT_TYPE	定义 RPMsg Endpoint 地址所属类型的宏
EPT_ADDR_MACRO	定义 RPMsg Endpoint 地址的宏。
RPMSG_QUERY_STAGE	查询从片当前所处的启动阶段
RPMSG_STAGE_RTOS	表示从片当前处于 RTOS 阶段
RPMSG_STAGE_LINUX	表示从片当前处于 Linux kernel 阶段
RPMSG_STAGE_APP	表示当前从片 APP 已经 ready
RPMSG_BOOTUP_FLAG_WAIT_APP	表示需要等到从片 APP ready 后，才返回
struct ss_rpmsg_endpoint_info	定义 RPMsg Endpoint 属性信息，如地址信息。
struct ss_rpmsg_devices_info	定义连接设备查询条件的结构体
struct ss_rpmsg_query_info	定义设备信息查询条件的结构体
struct ss_rpmsg_image_info	定义 image 下载信息的结构体
struct ss_rpmsg_bootup_info	通知从片开始启动 IOCTL 接口的参数结构体，含有启动入口和 bootargs 等信息
struct ss_rpmsg_bootup_done	定义 bootup 完成通知目标的结构体

10.1. RPMSG_MODE_RC_PORT

➤ 说明

表示 target 所在的 RPMsg bus 是长在本机 RC 口上的。

➤ 定义

```
#define RPMSG_MODE_RC_PORT 0
```

➤ 成员

无

※ 注意事项

无

➤ 相关数据类型及接口

无

10.2. RPMSG_MODE_EP_PORT

➤ 说明

表示 target 所在的 RPMsg bus 是长在本机 EP 口上的。

➤ 定义

```
#define RPMSG_MODE_EP_PORT 1
```

➤ 成员

无

※ 注意事项

无

➤ 相关数据类型及接口

无

10.3. RPMSG_MODE_UNKNOWN

➤ 说明

表示未知的 RPMsg bus 模式。

➤ 定义

```
#define RPMSG_MODE_UNKNOWN 3
```

➤ 成员

无

※ 注意事项

无

➤ 相关数据类型及接口

无

10.4. EPT_TYPE_CUSTOMER

➤ 说明

表示用户使用的(非 Sigmastar 使用)RPMsg Endpoint 地址。

➤ 定义

```
#define EPT_TYPE_CUSTOMER    0x1
```

➤ 成员

无

※ 注意事项

无

➤ 相关数据类型及接口

无

10.5. EPT_TYPE_SIGMASTAR

➤ 说明

表示非用户使用的(Sigmastar 使用)RPMsg Endpoint 地址。

➤ 定义

```
#define EPT_TYPE_SIGMASTAR    0x0
```

➤ 成员

无

※ 注意事项

用户进行应用开发时，不得使用这种类型的 RPMsg Endpoint。

➤ 相关数据类型及接口

无

10.6. EPT_TYPE

➤ 说明

定义 RPMsg Endpoint 地址所属类型的宏。

➤ 定义

```
#define EPT_TYPE(x)          ((x & 0x1) << 30)
```

➤ 成员

无

※ 注意事项

无

➤ 相关数据类型及接口

[EPT_TYPE_CUSTOMER](#)
[EPT_TYPE_SIGMASTAR](#)

10.7. EPT_ADDR_MACRO

- 说明
定义 RPSMsg Endpoint 地址的宏。
- 定义
`#define EPT_ADDR_MACRO(t, c) (EPT_TYPE(t) | (c & 0x3ffffff))`
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
[EPT_TYPE](#)
[EPT_TYPE_CUSTOMER](#)
[EPT_TYPE_SIGMASTAR](#)
[SS_RPSMSG_CREATE_EPT_IOCTL](#)

10.8. RPSMSG_QUERY_STAGE

- 说明
[struct ss_rpsmsg_query_info](#) 结构体中，成员 `param` 的合法值之一，表示查询从片当前所处阶段。
- 定义
`#define RPSMSG_QUERY_STAGE (0x1)`
- 成员
无
- ※ 注意事项
这个参数只支持 `mode` 为 [RPSMSG_MODE_RC_PORT](#) 对应的 target device。
- 相关数据类型及接口
[SS_RPSMSG_QUERY_IOCTL](#)

10.9. RPSMSG_STAGE_RTOS

- 说明
[RPSMSG_QUERY_STAGE](#) 查询的返回值之一，表示从片处于 RTOS 阶段。

- 定义
#define RPMSG_STAGE_RTOS (0x1))
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
[RPMSG_QUERY_STAGE](#)
[SS RPMSG_QUERY_IOCTL](#)

10.10. RPMSG_STAGE_LINUX

- 说明
[RPMSG_QUERY_STAGE](#) 查询的返回值之一，表示从片处于 Linux kernel 阶段。
- 定义
#define RPMSG_STAGE_LINUX (0x2)
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
[RPMSG_QUERY_STAGE](#)
[SS RPMSG_QUERY_IOCTL](#)

10.11. RPMSG_STAGE_APP

- 说明
[RPMSG_QUERY_STAGE](#) 查询的返回值之一，表示从片上 APP 已经准备好。
- 定义
#define RPMSG_STAGE_APP (0x3)
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口

[RPMMSG_QUERY_STAGE](#)
[SS RPMMSG_QUERY_IOCTL](#)

10.12. [RPMMSG_BOOTUP_FLAG_WAIT_APP](#)

- 说明
[struct ss_rpmsg_bootup_info](#) 接口中 flags 可以设置的 flag 之一，表示 [SS RPMMSG_BOOTUP_IOCTL](#) 将会等待从片 userspace app 调用了 [SS RPMMSG_BOOTUP_DONE_IOCTL](#) 接口后才会返回。
- 定义
#define RPMMSG_BOOTUP_FLAG_WAIT_APP (0x1)
- 成员
无
- ※ 注意事项
无
- 相关数据类型及接口
[SS RPMMSG_BOOTUP_IOCTL](#)
[SS RPMMSG_BOOTUP_DONE_IOCTL](#)

10.13. [struct ss_rpmsg_endpoint_info](#)

- 说明
定义创建的 RPMsg Endpoint 属性结构体。
- 定义
struct ss_rpmsg_endpoint_info {
 char name[32];
 __u32 src;
 __u32 dst;
 __u32 id;
 __u32 mode;
 __u16 target_id;
};

➤ 成员

成员名称	描述
name	RPMsg Endpoint 的名字
src	RPMsg Endpoint 的本地地址，远端要发信息给这个 Endpoint，其 dst 地址就要填写这个值。
dst	RPMsg Endpoint 的信息发送目标地址，write Endpoint 时，信息会被发送到这个地址。

成员名称	描述
id	ioctl 的返回值，表示 RPMsg Endpoint 节点/dev/rpmsgX 后面 X 的值。
mode	Target 所在 RPMsg bus 的模式，其值为： RPMSG_MODE_RC_PORT 、 RPMSG_MODE_EP_P ORT
target_id	其值的含义随 mode 的值而定 RPMSG_MODE_RC_PORT ，则 target_id 为目标 Chip 的编号 RPMSG_MODE_EP_PORT ，则 target_id 为 PortId

※ 注意事项
无。

- 相关数据类型及接口
[RPMSG_MODE_RC_PORT](#)
[RPMSG_MODE_EP_PORT](#)
[EPT_TYPE_CUSTOMER](#)
[EPT_TYPE_SIGMASTAR](#)
[EPT_TYPE](#)
[EPT_ADDR_MACRO](#)

10.14. struct ss_rpmsg_devices_info

- 说明
定义 devices 列表的查询条件。

- 定义
- ```
struct ss_rpmsg_devices_info {
 __u32 mode;
 __u32 count;
 __u64 buffer;
};
```

- 成员

| 成员名称   | 描述                                                                                                           |
|--------|--------------------------------------------------------------------------------------------------------------|
| mode   | Target 所在 RPMsg bus 的模式，其值为：<br><a href="#">RPMSG_MODE_RC_PORT</a> 、 <a href="#">RPMSG_MODE_EP_P<br/>ORT</a> |
| count  | 作为输入，表示 buffer 最多可以存几个 target_id，<br>作为输出，返回值表示 buffer 里有几个<br>target_id。                                    |
| buffer | Userspace buffer 地址，kernel 会往里面填写当前<br>已连接 Chips 的 target_id 列表。                                             |

※ 注意事项  
无。

- 相关数据类型及接口
  - [RPMSG\\_MODE\\_RC\\_PORT](#)
  - [RPMSG\\_MODE\\_EP\\_PORT](#)

10.15. struct ss\_rpmsg\_query\_info

- 说明

定义查询参数。

- 定义

```
struct ss_rpmsg_query_info {
 __u32 mode;
 __u16 target_id;
 __u32 param;
 __u64 value;
};
```

- 成员

| 成员名称      | 描述                                                                                                                                                                  |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mode      | Target 所在 RPMsg bus 的模式，其值为：<br><a href="#">RPMSG_MODE_RC_PORT</a> 、 <a href="#">RPMSG_MODE_EP_P<br/>ORT</a>                                                        |
| target_id | 其值的含义随 mode 的值而定<br><a href="#">RPMSG_MODE_RC_PORT</a> ，则 target_id 为目标<br>Chip 的编号<br><a href="#">RPMSG_MODE_EP_PORT</a> ，则 target_id 为 PortId                     |
| param     | 指定查询参数，可取值：<br><a href="#">RPMSG_QUERY_STAGE</a>                                                                                                                    |
| value     | 指定查询参数的返回值，<br><a href="#">RPMSG_QUERY_STAGE</a> 对应的值为：<br><a href="#">RPMSG_STAGE_RTOS</a><br><a href="#">RPMSG_STAGE_LINUX</a><br><a href="#">RPMSG_STAGE_APP</a> |

- ※ 注意事项
  - 无。

- 相关数据类型及接口
  - [SS\\_RPMSG\\_QUERY\\_IOCTL](#)
  - [RPMSG\\_QUERY\\_STAGE](#)
  - [RPMSG\\_STAGE\\_RTOS](#)
  - [RPMSG\\_STAGE\\_LINUX](#)
  - [RPMSG\\_STAGE\\_APP](#)

## 10.16. struct ss\_rpmsg\_image\_info

### ➤ 说明

定义 image 下载信息的结构体。

### ➤ 定义

```
struct ss_rpmsg_image_info {
 __u32 mode;
 __u16 target_id;
 char name[32];
 __u64 reserved;
 __u64 addr_phys;
 __u64 load_addr_phys;
};
```

### ➤ 成员

| 成员名称           | 描述                                                                                  |
|----------------|-------------------------------------------------------------------------------------|
| mode           | Target 所在 RPMsg bus 的模式，只支持：<br><a href="#">RPMSG_MODE_RC_PORT</a>                  |
| target_id      | 其值的含义随 mode 的值而定<br><a href="#">RPMSG_MODE_RC_PORT</a> ，则 target_id 为目标<br>Chip 的编号 |
| name           | image 名称，最大不能超过 32 个 bytes(含 '\0')。                                                 |
| reserved       | reserved 成员                                                                         |
| addr_phys      | image buffer 的起始物理地址                                                                |
| load_addr_phys | image 在从片上的存储物理地址                                                                   |

### ※ 注意事项

无。

### ➤ 相关数据类型及接口

[SS\\_RPMSG\\_LOAD\\_IMAGE\\_IOCTL](#)

## 10.17. struct ss\_rpmsg\_bootup\_info

### ➤ 说明

定义 bootup 信息的结构体。

### ➤ 定义

```
struct ss_rpmsg_bootup_info {
 __u32 mode;
 __u16 target_id;
 __u32 flags;
 __u64 entry;
 __u64 bootargs;
};
```

### ➤ 成员

| 成员名称 | 描述 |
|------|----|
|------|----|

|           |                                                                                     |
|-----------|-------------------------------------------------------------------------------------|
| mode      | Target 所在 RPMsg bus 的模式，只支持：<br><a href="#">RPMSG_MODE_RC_PORT</a> 、                |
| target_id | 其值的含义随 mode 的值而定<br><a href="#">RPMSG_MODE_RC_PORT</a> ，则 target_id 为目标<br>Chip 的编号 |
| flags     | <a href="#">RPMSG_BOOTUP_FLAG_WAIT_APP</a>                                          |
| entry     | Bootup 的跳转入口地址                                                                      |
| bootargs  | '\0'结尾的 kernel 启动参数字符串                                                              |

※ 注意事项  
无。

➤ 相关数据类型及接口  
[SS\\_RPMSG\\_BOOTUP\\_IOCTL](#)

10.18. struct ss\_rpmsg\_bootup\_done

➤ 说明  
定义 bootup 完成通知目标的结构体

➤ 定义

```
struct ss_rpmsg_bootup_done {
 __u32 mode;
 __u16 target_id;
};
```

➤ 成员

| 成员名称      | 描述                                                                           |
|-----------|------------------------------------------------------------------------------|
| mode      | Target 所在 RPMsg bus 的模式，只支持：<br><a href="#">RPMSG_MODE_EP_PORT</a>           |
| target_id | 其值的含义随 mode 的值而定<br><a href="#">RPMSG_MODE_EP_PORT</a> ，则 target_id 为 PortId |

※ 注意事项  
无。

➤ 相关数据类型及接口  
[SS\\_RPMSG\\_BOOTUP\\_DONE\\_IOCTL](#)